

Petrea Bogdan-Vasile 342C3

## TEMA 2 ML

Pentru acest proiect ne dorim să analizăm un set de imagini color (RGB) care surprind diverse obiecte și ființe, cu scopul de a realiza clasificarea automată în una dintre 10 clase: airplane, truck, bird, car, cat, deer, dog, horse, monkey, ship. Pornind de la această problemă de recunoaștere vizuală, urmăm să explorăm și să comparăm mai multe arhitecturi de rețele neurale, de la un model simplu de tip MLP (care primește imaginea aplatizată) până la modelele CNN (capabile să surprindă structura spațială a imaginilor). Prin această comparație, evaluăm ce alegeri de arhitectură și ce tehnici (ex. regularizare/augmentare) contribuie la îmbunătățirea performanței și a capacității de generalizare pe date nevăzute.

### EDA

Dataset-ul "Imagebits" este format din imagini color (RGB) cu rezoluția fixă 96×96 pixeli și conține 10 clase: airplane, bird, car, cat, deer, dog, horse, monkey, ship, truck. Conform descrierii setului de date, există 800 de imagini de antrenare și 500 de imagini de test pentru fiecare clasă, adică 8.000 imagini în setul de antrenare și 5.000 imagini în setul de test (13.000 în total). În implementarea noastră, dataset-ul a fost disponibil inițial doar în două subseturi (train și test), astfel că am realizat suplimentar un split stratificat din setul de antrenare pentru a obține un set de validare (val). Acest lucru ne permite să monitorizăm corect performanța în timpul antrenării, să comparăm configurații (de exemplu, cu și fără augmentare) și să păstrăm setul de test exclusiv pentru evaluarea finală, fără a introduce bias în selecția modelului.

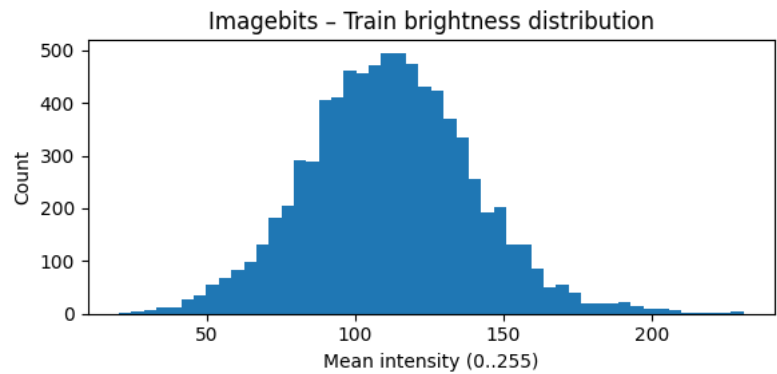
Am început analiza EDA printr-un scan de integritate al dataset-ului, pentru a verifica dacă există imagini corupte sau care nu pot fi citite (o problemă frecventă care poate opri antrenarea sau introduce erori greu de identificat). Scanarea a confirmat că toate fișierele sunt valide: nu au fost găsite imagini corupte/necitabile nici în subsetul de train, nici în subsetul de test. În plus, am verificat și rezoluțiile imaginilor, observând că toate au aceeași dimensiune, 96×96, lucru care confirmă informațiile pe care le-am primit în enunț. Acest lucru ne va ajuta prin simplificarea pipeline-ului de preprocesare.

| Split | Total imagini scanate | OK (citibile) | Corupte / necitibile | Rezoluții unice | Rezoluție (dominantă) | Nr. imagini la 96×96 |
|-------|-----------------------|---------------|----------------------|-----------------|-----------------------|----------------------|
| Train | 8000                  | 8000          | 0                    | 1               | 96×96                 | 8000                 |
| Test  | 5000                  | 5000          | 0                    | 1               | 96×96                 | 5000                 |

Am calculat statisticile de culoare pe setul de antrenare (media și deviația standard pe canalele R, G, B) pentru a folosi o normalizare adaptată dataset-ului în pipeline-ul de preprocesare. Normalizarea cu aceste valori stabilizează antrenarea (scale consistente ale intrărilor) și poate accelera convergența. În plus, am analizat distribuția luminozității (intensitatea medie per imagine), observând o distribuție aproximativ normală, centrată în jurul unei valori medii moderate, dar cu o deviație suficient de mare încât indică variații de iluminare/contrast între imagini. Această variație justifică includerea unor augmentări de tip ColorJitter (brightness/contrast) în experimentul cu augmentare, pentru a crește robustețea modelului la condiții diferite de iluminare.

| Statistică (train)      | Valoare                              |
|-------------------------|--------------------------------------|
| # imagini folosite      | 8000                                 |
| Mean RGB                | [0.4472306, 0.43964246, 0.4049573]   |
| Std RGB                 | [0.26056457, 0.25666147, 0.26997384] |
| Brightness mean (0–255) | 111.68                               |
| Brightness std (0–255)  | 28.18                                |

Distribuția luminozității (train) arată variabilitate moderată a intensității medii între imagini, motiv pentru care augmentările de tip brightness/contrast pot îmbunătăți generalizarea.



### Variabilitate intra- și inter-clasă (exemple vizuale)

Exemplele vizuale pe clase evidențiază atât **variabilitate intra-clasă**, cât și **variabilitate inter-clasă**. În interiorul aceleiași clase (ex. *airplane*, *truck*, *dog*), imaginile pot diferi semnificativ prin **fundal**, **unghi de captură**, **scală/zoom**, **poziția obiectului** și **condiții de iluminare** (de la scene luminoase la cadre întunecate sau cu contrast puternic). Această diversitate impune modele capabile să învețe trăsături robuste și invariante la transformări, motiv pentru care arhitecturile de tip **CNN** sunt mai potrivite decât un MLP simplu.

În același timp, există și **similarități inter-clasă** care pot induce confuzii: clasele de animale (*cat*, *dog*, *deer*, *horse*, *monkey*) împart texturi și forme globale asemănătoare, mai ales la rezoluția 96×96, iar diferențele fine pot fi greu de surprins. Acest lucru justifică utilizarea **regularizării** și a **augmentării** (ex. flip, rotații mici, ajustări de

luminozitate/contrast), pentru a îmbunătăți generalizarea și a reduce supraînvățarea pe tipare de fundal.



### Echilibrul claselor și particularități relevante pentru antrenare

Dataset-ul este **echilibrat pe clase**, având același număr de exemple pentru fiecare categorie (conform descrierii: 800 imagini/clasă în train și 500 imagini/clasă în test). Acest aspect este important deoarece reduce riscul ca modelul să devină părtinitor către o clasă majoritară și face ca metrice precum acuratețea și macro-F1 să fie ușor de interpretat. În plus, imaginile au o rezoluție mică (96×96), ceea ce limitează detaliile fine (mai ales între clase vizual similare, precum animalele) și poate conduce la confuzii. Variabilitatea intra-clasă (fundaluri diferite, poziții, iluminare) și variațiile de luminozitate/contrast observate în EDA justifică folosirea **normalizării** (cu mean/std calculate pe train) și testarea **augmentării** (flip/rotație/ColorJitter) pentru a îmbunătăți robustețea și generalizarea pe setul de test.

#### Implicații pentru antrenare:

- date echilibrate → nu sunt necesare ponderi pe clase;
- rezoluție mică → modele CNN sunt preferate față de MLP;
- variații de iluminare → normalizare + augmentări de culoare pot ajuta;
- clase similare (animale) → confuzii așteptate în matricea de confuzie.

### Concluzie EDA

În urma analizei exploratorii, dataset-ul Imagebits este consistent și potrivit pentru antrenare: nu conține fișiere corupte, toate imaginile au aceeași rezoluție (96×96), iar statisticile de intensitate confirmă necesitatea normalizării și justifică augmentările de tip iluminare/contrast. În plus, exemplele vizuale evidențiază atât variabilitate intra-clasă, cât și similarități inter-clasă (în special între clasele de animale), ceea ce sugerează că un model care exploatează structura spațială (CNN) va fi mai potrivit decât un model care aplatizează imaginea (MLP).

### Împărțirea datelor (train/val/test)

Dataset-ul este disponibil inițial în două subseturi: **train** și **test**. Pentru a putea monitoriza performanța în timpul antrenării și a compara corect diferite configurații (ex. cu și fără augmentare), am realizat suplimentar un **split stratificat** din setul de antrenare pentru a obține un set de **validare (val)**. Split-ul stratificat păstrează proporția claselor în ambele subseturi, ceea ce este important într-un task multi-clasă pentru a evita un set de validare dezechilibrat. În această manieră, setul de **test** rămâne utilizat exclusiv pentru evaluarea finală, iar deciziile privind alegerea configurației se bazează pe performanța pe **val**, reducând riscul de "leakage" și supraestimare a performanței.

| Split | Total | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  |
|-------|-------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Train | 6400  | 640 | 640 | 640 | 640 | 640 | 640 | 640 | 640 | 640 | 640 |
| Val   | 1600  | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 | 160 |

### Transformări, dataset-uri și DataLoaders

Pentru preprocesare am folosit o normalizare adaptată dataset-ului, folosind valorile **MEAN** și **STD** calculate pe setul de antrenare (per-canal RGB). Am definit trei pipeline-uri de transformări: (1) **train\_tfms\_base** – transformări de bază (Resize → ToTensor → Normalize), (2) **train\_tfms\_aug** – transformări de antrenare cu augmentare (Resize + `RandomHorizontalFlip(p=0.5)`, `RandomRotation(10°)`, `ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1)` → ToTensor → Normalize) și (3) **eval\_tfms** – transformări de evaluare identice cu baseline (fără augmentare), utilizate pentru validare și test. Pentru a compara corect baseline vs augmentare, am creat două versiuni ale dataset-ului de train (`ImageFolder`) cu transformări diferite, dar am aplicat **aceiași indici** de split (`Subset` cu `train_idx/val_idx`) astfel încât seturile efective de train/val să conțină exact aceleași imagini. Încărcarea în batch-uri este realizată prin `DataLoader`, cu `shuffle=True` doar pentru train și `shuffle=False` pentru val/test, iar `pin_memory` este activat doar atunci când antrenarea rulează pe GPU (CUDA), pentru eficiență.

**Justificare.** Normalizarea cu **MEAN/STD calculate pe dataset** (în locul unor valori "generice") aduce intrările pe o scară comparabilă pe fiecare canal RGB, ceea ce stabilizează distribuția activărilor și face optimizarea mai eficientă (convergență mai stabilă și sensibilitate mai mică la variații de luminozitate/contrast). În același timp, **augmentarea este aplicată exclusiv pe setul de antrenare**, deoarece scopul ei este să crească diversitatea datelor văzute de model și să îmbunătățească robustețea (invariantă la flip/rotații mici și variații de culoare). Pentru validare și test păstrăm doar transformările de bază, astfel încât metricele raportate să reflecte performanța pe exemple "naturale", fără a introduce zgomot artificial în evaluare și fără a altera comparabilitatea între configurații.

### Modele utilizate

Pentru a evidenția impactul arhitecturii asupra performanței în clasificarea de imagini, am implementat două familii de modele: un **MLP** (baseline) și un **CNN** (convoluțional).

- **MLP (baseline).** Imaginea RGB de dimensiune **3×96×96** este **aplatizată** într-un vector și trecută prin două straturi fully-connected cu dimensiuni ascunse **1024** și **512**, fiecare urmat de **ReLU** și **Dropout(0.3)**. Ultimul strat fully-connected proiectează către cele 10 clase. Acest model servește drept referință, însă pierde structura spațială a imaginii prin operația de aplatizare.
- **CNN (simplu).** Am definit un CNN compus din **4 blocuri** repetate de forma **Conv2D → BatchNorm → ReLU → MaxPool**, cu lățimea canalelor crescătoare **3→32→64→128→256**. Blocurile reduc progresiv rezoluția spațială și extrag caracteristici locale (muchii, texturi, forme). Reprezentarea finală este aplatizată și trecută

printr-un clasificator fully-connected **Linear** → **ReLU** → **Dropout** → **Linear**, cu strat intermediar de **512** neuroni. În mod natural, CNN-ul este mai potrivit pentru imagini deoarece păstrează și exploatează relațiile spațiale dintre pixeli.

#### Clarificare: cum funcționează MLP și CNN (descriere generală)

##### MLP (Multi-Layer Perceptron)

Un **MLP** este o rețea neuronală formată din straturi *fully-connected* (dense). Pentru imagini, primul pas este de obicei **aplatizarea** (flatten) matricei de pixeli într-un singur vector. Fiecare neuron dintr-un strat dense „vede” simultan **toate** valorile de intrare și învață o combinație liniară a acestora, urmată de o funcție de activare neliniară (ex. **ReLU**) care permite modelului să învețe relații complexe.

În practică, pentru clasificare:

1. Imaginea este transformată într-un vector.
2. Straturile dense aplică transformări succesive (proiecții în spații latente).
3. Stratul final produce scoruri (logits) pentru fiecare clasă, iar **softmax** (implicit în CrossEntropyLoss) transformă aceste scoruri în probabilități.

**Limitarea principală** pentru imagini este că, după aplatizare, se pierde structura spațială (vecinătăți, poziții relative). Astfel, MLP-ul trebuie să „reînvețe” implicit relațiile spațiale doar din combinații de pixeli, ceea ce îl face de regulă mai ineficient și mai sensibil la variații (translații, rotații, schimbări de iluminare).

##### CNN (Convolutional Neural Network)

Un **CNN** este specializat pentru date cu structură spațială (imagini) și învață direct **caracteristici locale** prin operația de **convoluție**. În loc ca fiecare neuron să fie conectat la toți pixelii, un filtru (kernel) mic (ex.  $3 \times 3$ ) „scanează” imaginea și produce **feature maps** care detectează tipare simple la început (muchii, colțuri) și din ce în ce mai complexe în straturile mai adânci (texturi, părți de obiecte, forme).

Într-un CNN tipic:

1. **Convoluțiile** extrag tipare locale folosind filtre învățate din date (parametri partajați pe întreaga imagine).
2. **ReLU** introduce neliniaritate și permite învățarea de reprezentări complexe.
3. **Pooling**-ul (ex. MaxPool) reduce rezoluția spațială, crește robustețea la mici translații și reduce costul computațional.
4. La final, reprezentarea este aplatizată și trecută printr-un **clasificator fully-connected** care decide clasa pe baza caracteristicilor învățate.

Avantajul major al CNN-urilor este că păstrează structura spațială, folosesc **partajarea parametrilor** (mai puțini parametri decât un MLP echivalent) și învață reprezentări **ierarhice** (de la trăsături simple la trăsături complexe), fiind în general mai potrivite pentru clasificarea de imagini.

## Setup de antrenare și evaluare (comun pentru toate modelele)

Toate modelele au fost antrenate într-un cadru unitar pentru a asigura o comparație corectă între arhitecturi. Pentru clasificare multi-clasă am folosit **CrossEntropyLoss**, iar optimizarea s-a realizat cu **Adam** și regularizare prin **weight decay**. La fiecare epocă am calculat și salvat:

- **Train/Val Loss**
- **Train/Val Accuracy**

Selecția modelului final s-a făcut printr-o strategie de tip **best checkpoint**: dacă **val\_acc** crește, se salvează **state\_dict**-ul corespunzător. La final se încarcă automat starea cu **best\_val\_acc**. Astfel, setul de **test** este utilizat exclusiv pentru evaluarea finală și nu influențează alegerea hiperparametrilor sau a modelului.

#### Configurația experimentelor (CFG)

- **VAL\_RATIO** = 0.2 (split de validare)
- **SEED** = 42 (reproductibilitate; `cuda.deterministic=True, cuda.benchmark=False`)
- **IMG\_SIZE** = 96
- **BATCH\_SIZE** = 128
- **EPOCHS** = 15
- Optimizator: **Adam**
- **LR** =  $1e-3$
- **WEIGHT\_DECAY** =  $1e-4$
- Data loading: **NUM\_WORKERS** = 2, **PIN\_MEMORY** = True (pin\_memory activ doar pe GPU)
- Mixed precision: activată automat pe GPU (`use_amp=True` dacă `cuda` este disponibil)

Sunt incluse, pentru fiecare model, curbele **Train/Val Loss** (pe același grafic) și **Train/Val Accuracy** (pe un grafic separat). De asemenea, pentru comparația între configurații am raportat metrici în tabel (**Accuracy** și **Macro-F1**).

**Justificarea alegerii optimizatorului (Adam) și a learning rate-ului.** Am ales **Adam** cu **LR=1e-3** deoarece, în practică, oferă **convergență stabilă** și rapidă pentru rețele cu multe parametre, reducând sensibilitatea la scalarea gradientului față de optimizatori mai simpli. Ținta mea a fost să obțin o bază reproductibilă și comparabilă între experimente (MLP vs CNN, cu/fără augmentare), fără a introduce variații suplimentare din schimbarea optimizatorului. În experimentele prezentate nu am folosit scheduler (learning rate constant).

## Experimente (Imagebits)

Pentru dataset-ul **Imagebits** am rulat trei experimente controlate, menite să izoleze separat efectul arhitecturii și al augmentării:

1. **MLP baseline** – model de referință (fără mecanisme explicite de exploatare a structurii spațiale).
2. **CNN baseline (no augmentation)** – aceeași configurație de antrenare, dar cu o arhitectură convoluțională, pentru a cuantifica avantajul folosirii convoluțiilor.
3. **CNN augmented** – aceeași arhitectură ca în (2), dar antrenată cu augmentări aplicate **doar pe setul de train** (validarea/testul rămân neaugmentate), pentru a evalua efectul regularizării prin diversificarea datelor.

Augmentări folosite (train only)

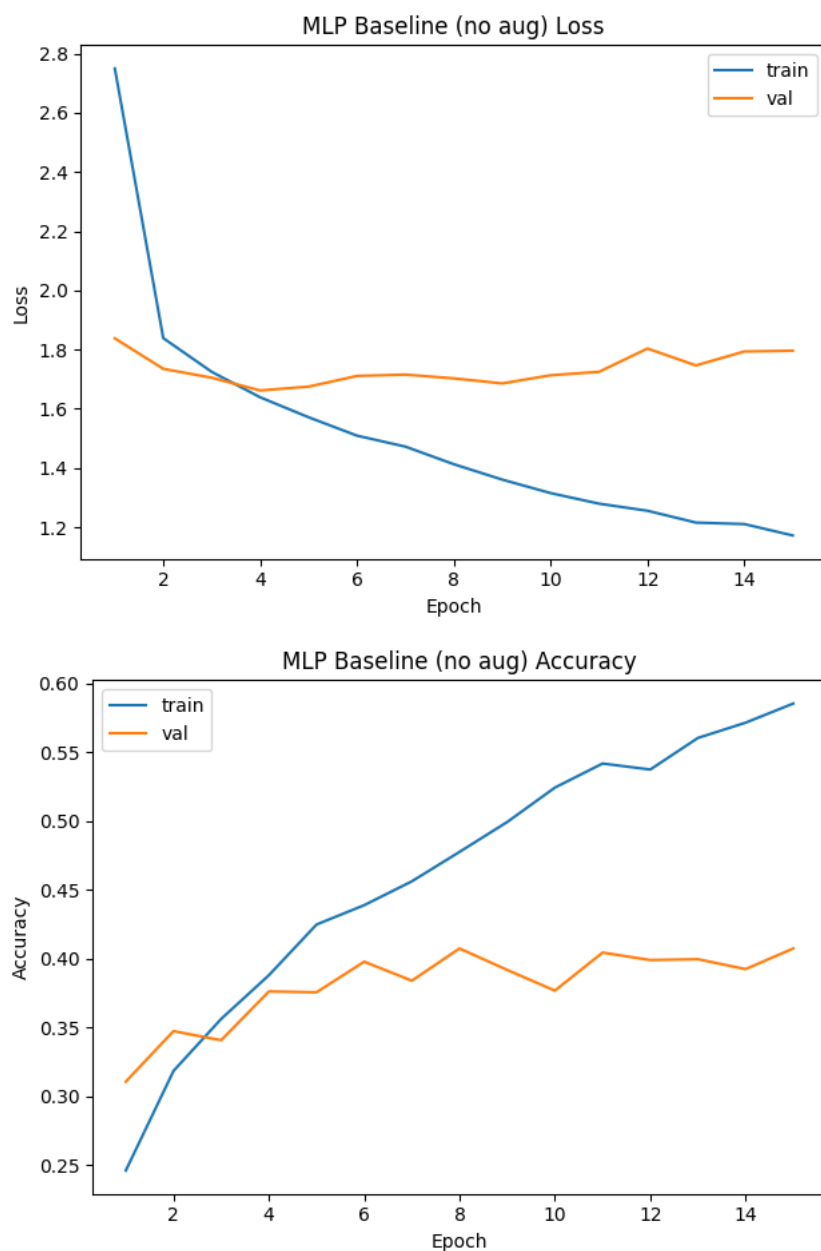
- **RandomHorizontalFlip** cu  $p=0.5$
- **RandomRotation** până la  $\pm 10^\circ$
- **ColorJitter**:  $\text{brightness}=0.1$ ,  $\text{contrast}=0.1$ ,  $\text{saturation}=0.1$

Toate cele trei experimente folosesc aceeași configurație de antrenare ( $\text{EPOCHS}=15$ ,  $\text{BATCH\_SIZE}=128$ ,  $\text{LR}=1\text{e-}3$ ,  $\text{WEIGHT\_DECAY}=1\text{e-}4$ ), astfel încât diferențele observate să reflecte strict arhitectura și/sau augmentarea.

## Analiza experimentelor și a curbelor de învățare (Imagebits)

În această secțiune interpretăm curbele **Train/Val Loss** și **Train/Val Accuracy** pentru cele trei experimente rulate (MLP baseline, CNN baseline, CNN + augmentare). Pentru toate modelele, selecția finală s-a făcut prin **best checkpoint** (modelul salvat la epoca cu  $\text{val\_acc}$  maximă), astfel încât concluziile de performanță să nu fie influențate de eventuale epoci finale în care apare overfitting.

### 1) MLP baseline (fără augmentare)



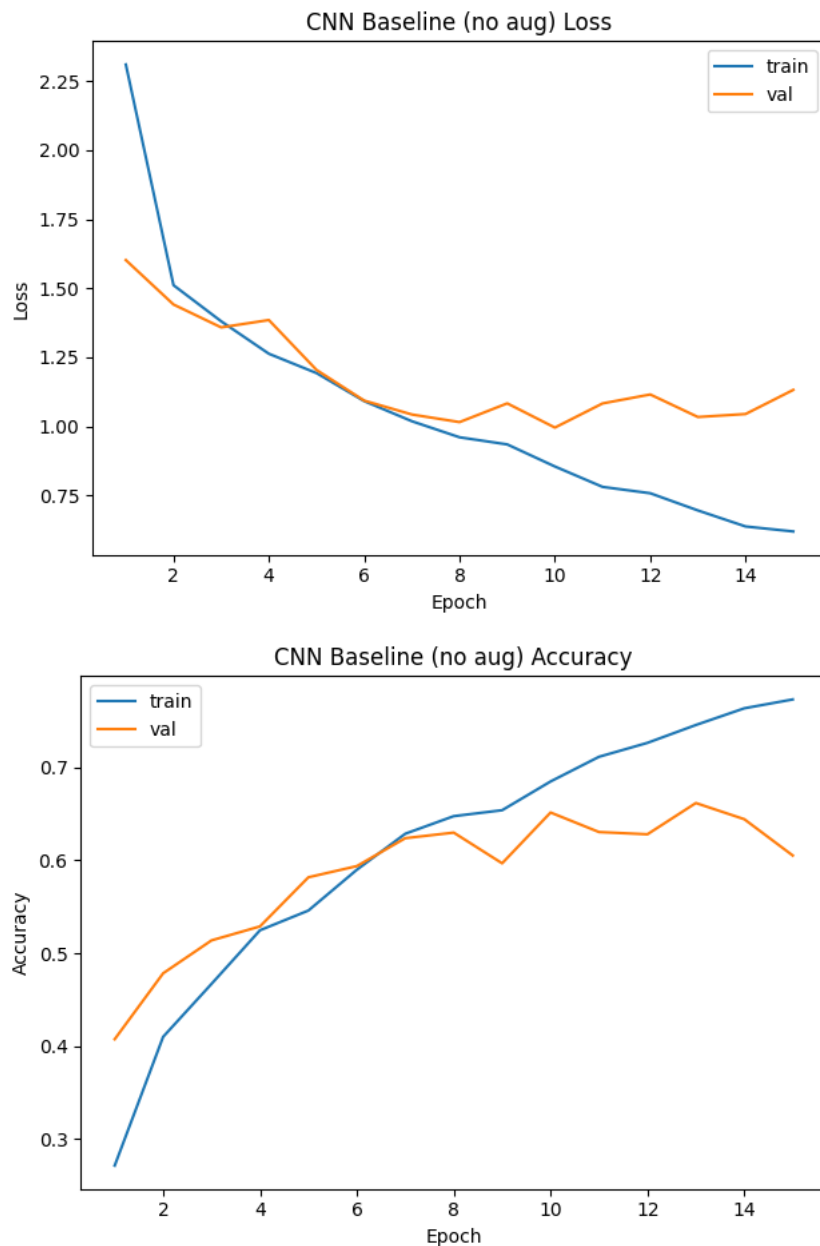
#### Observații din curbe:

- **Train loss** scade constant pe parcursul epocilor, ceea ce indică faptul că modelul reușește să se potrivească pe datele de antrenare.
- **Val loss** scade ușor la început, apoi tinde să crească/oscileze în a doua parte a antrenării. Acest comportament sugerează **overfitting**: modelul continuă să se îmbunătățească pe train, dar nu mai generalizează mai bine pe validare.

- **Train accuracy** crește constant, însă **Val accuracy** se plafonează relativ devreme și rămâne în jurul unei valori moderate, cu fluctuații.

**Interpretare:** MLP-ul este un baseline util, dar pentru imagini performează limitat deoarece aplatizează intrarea și pierde structura spațială (vecinătăți, forme). Prin urmare, crește rapid diferența dintre train și val, semn clar de generalizare slabă.

## 2) CNN baseline (fără augmentare)

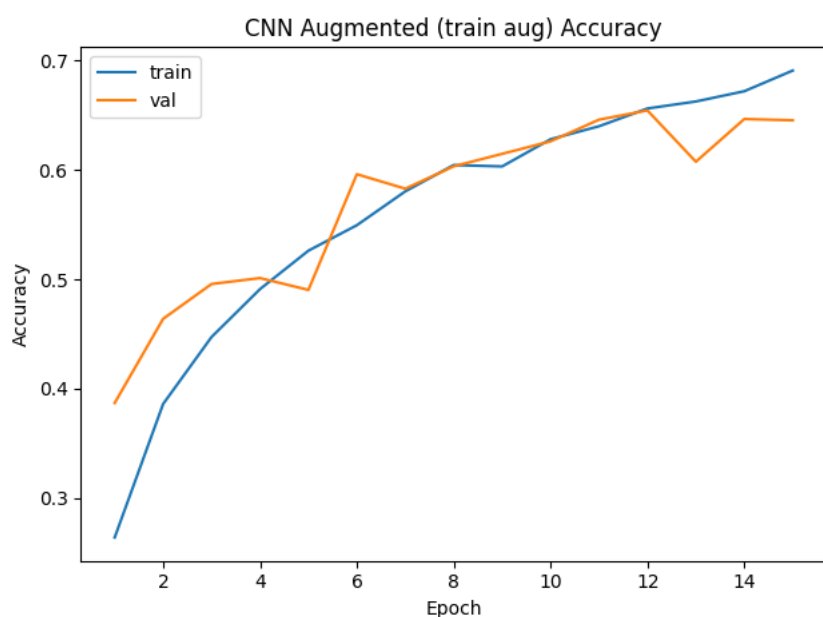
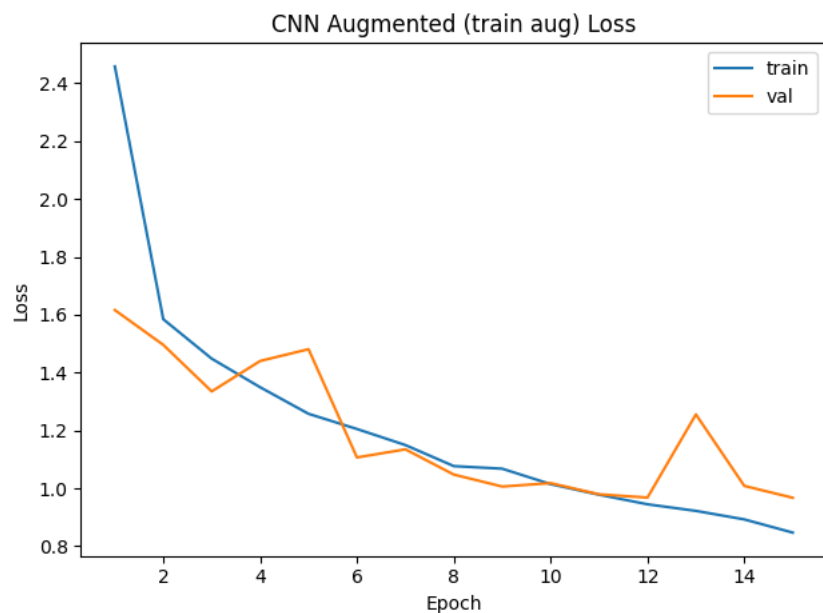


### Observații din curbe:

- Comparativ cu MLP, **convergența este mai bună**: train loss scade mai mult și mai rapid, iar train accuracy ajunge la valori considerabil mai ridicate.
- **Val accuracy** este semnificativ mai mare decât la MLP, ceea ce confirmă avantajul arhitecturii convoluționale în extragerea de caracteristici relevante (muchii, texturi, forme).
- În partea a doua a antrenării, se vede o **separare** între train și val: train continuă să se îmbunătățească, în timp ce val loss începe să crească ușor iar val accuracy oscilează. Acesta este un semn tipic de **overfitting** după o anumită epocă.

**Interpretare:** CNN baseline oferă un salt clar de performanță față de MLP, deoarece păstrează structura spațială și învață reprezentări ierarhice. Totuși, fără augmentare, modelul poate începe să supraînvețe pe train după ce a extras deja caracteristicile utile.

## 3) CNN cu augmentare (augmentări doar pe train)

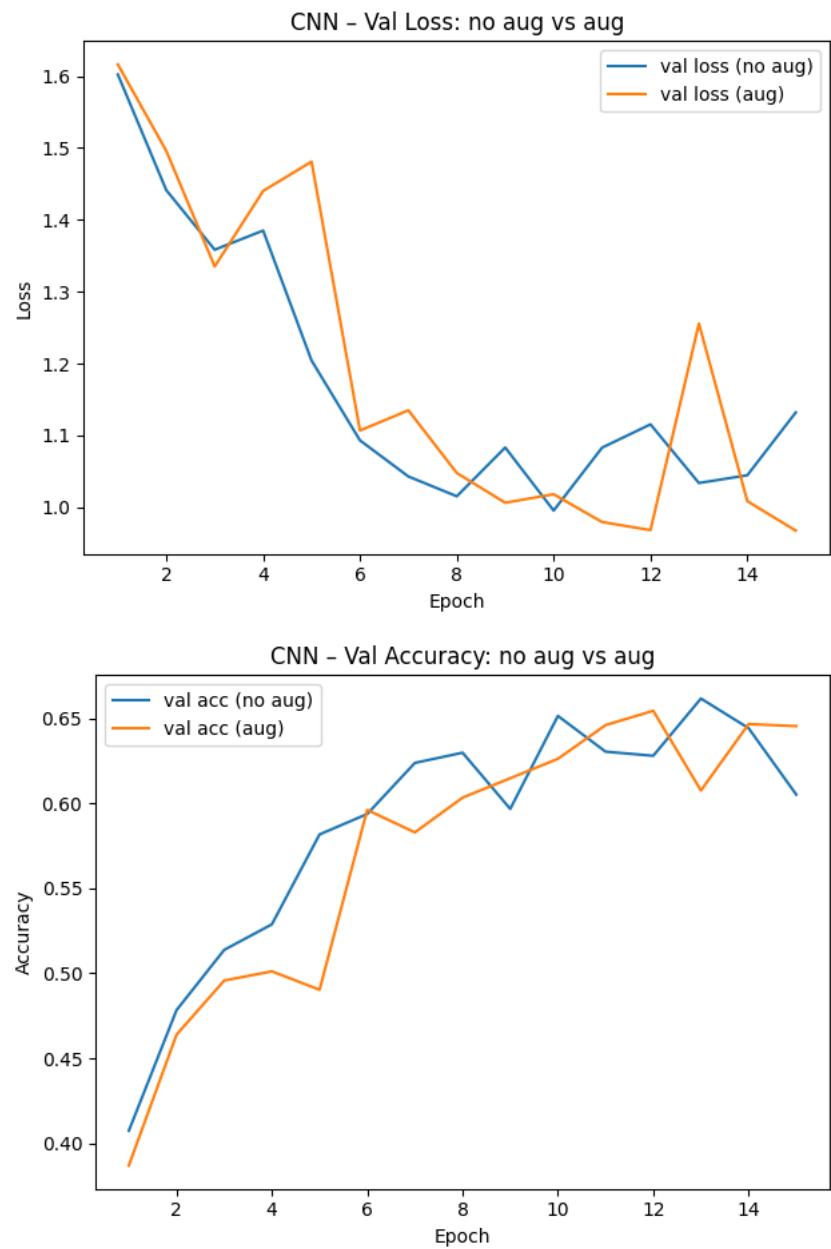


#### Observații din curbe:

- **Train accuracy** crește mai lent și rămâne mai mică decât la CNN no-aug (așteptat), deoarece augmentările fac problema „mai grea” pe train și acționează ca regularizare.
- **Val loss** tinde să fie mai stabil în a doua parte a antrenării (cu mici fluctuații), ceea ce indică o **generalizare mai bună** și un overfitting mai redus comparativ cu varianta fără augmentare.
- Se observă fluctuații punctuale în val (ex. un „spike” temporar al val loss), fenomen normal deoarece augmentarea introduce variabilitate, iar setul de validare rămâne fix.

**Interpretare:** Augmentarea reduce tendința de overfitting și îmbunătățește robustețea modelului la variații (orientare, iluminare, mici rotații). Chiar dacă train performance este mai mică, scopul augmentării este să mențină/îmbunătățească performanța pe date nevăzute.

#### 4) CNN fără aug vs CNN cu aug



Ce arată comparația:

- Pe **Val Loss**, varianta **cu augmentare** tinde să aibă o evoluție mai favorabilă în a doua parte a antrenării (în general, val loss mai mic sau mai stabil), sugerând o generalizare mai bună.
- Pe **Val Accuracy**, cele două variante sunt apropiate ca nivel, însă augmentarea ajută de obicei la **stabilitate** și la reducerea „gap”-ului train-val (modelul nu mai crește excesiv pe train în detrimentul val).

Concluzie din comparație:

- CNN** este net superior **MLP** pe acest task (extragere de caracteristici spațiale).
- Augmentarea** este utilă ca regularizare: chiar dacă nu crește dramatic vârful de acuratețe pe validare în fiecare rulare, reduce overfitting-ul și tinde să ofere un comportament mai robust.

**Impactul hiperparametrilor (observație).** Performanța este influențată puternic de hiperparametri precum **LR**, **weight\_decay**, **batch\_size** și dropout. În această etapă am menținut acești parametri **constanți** între experimente pentru a izola efectul arhitecturii (MLP vs CNN) și al augmentării. Totuși, curbele train/val sugerează că overfitting-ul poate fi redus prin ajustări de regularizare (ex. creșterea **weight\_decay**/dropout) sau printr-un **LR** mai mic / scheduler.

Rezultate finale pe setul de test (Imagebits)

În această secțiune vorbim despre performanța finală pe **setul de test**, utilizând **Accuracy** și **Macro-F1** (Macro-F1 este relevant deoarece evaluează echilibrat performanța pe clase). Evaluarea pe test a fost realizată **după** selecția checkpoint-ului optim pe baza **val\_acc** (*best checkpoint*), astfel încât setul de test să rămână utilizat exclusiv pentru raportarea finală.

Tabel comparativ (test)

| Model | Augmentare (train) | Test Accuracy | Test Macro-F1 |
|-------|--------------------|---------------|---------------|
|-------|--------------------|---------------|---------------|

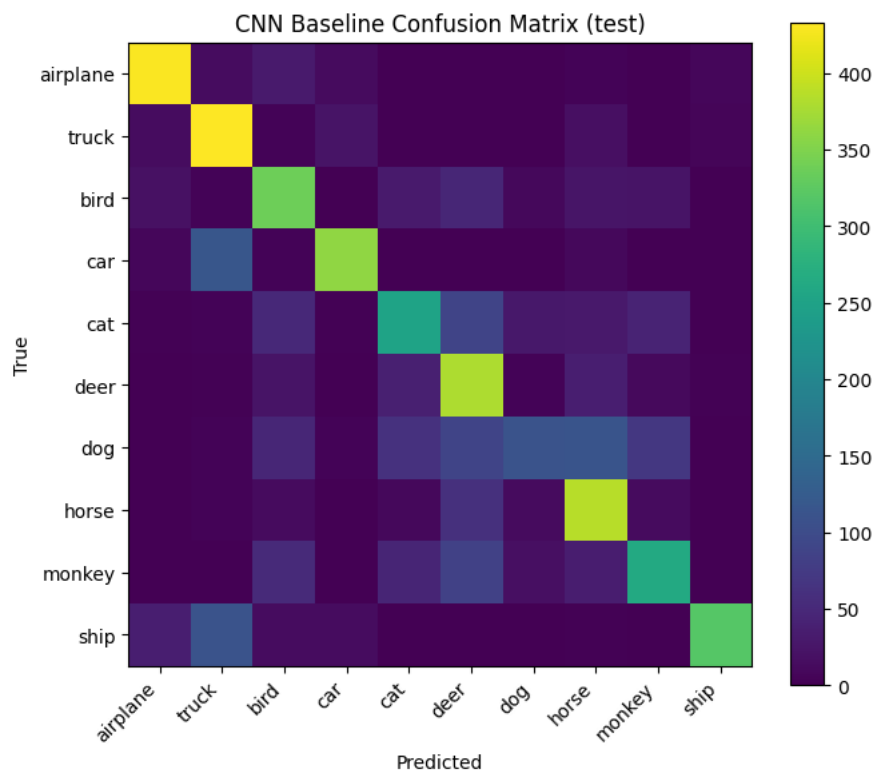
| Model         | Augmentare (train) | Test Accuracy | Test Macro-F1 |
|---------------|--------------------|---------------|---------------|
| MLP baseline  | nu                 | 0.3978        | 0.3879        |
| CNN baseline  | nu                 | 0.6544        | 0.6451        |
| CNN augmented | da                 | 0.6528        | 0.6453        |

Observații:

- Trecerea de la **MLP** la **CNN** aduce o îmbunătățire majoră a performanței pe test ( $\approx +0.26$  absolut la Accuracy), confirmând avantajul arhitecturilor convoluționale pentru imagini.
- Augmentarea** (în configurația curentă) nu produce o creștere clară a Accuracy pe test față de CNN baseline (diferențele sunt foarte mici), însă menține Macro-F1 la un nivel comparabil, sugerând un efect de regularizare fără schimbări semnificative ale performanței finale în această rulare.

Analiza erorilor: Matricea de confuzie (best CNN)

Pentru analiza detaliată a erorilor am selectat drept **best CNN** modelul cu **best\_val\_acc** maxim (în acest caz **CNN baseline**) și am raportat matricea de confuzie pe test.



Tipare importante observate

- Clase bine separate:** **airplane** și **ship** au predicții predominant pe diagonală, sugerând caracteristici vizuale distinctive.
- Confuzii între clase similare:** apar confuzii între **car** și **truck**, respectiv între animale cu aspect apropiat (ex. **cat** vs **dog**, **deer** vs **horse/monkey**), în special când fundalul sau poziția obiectului variază.
- Un caz notabil este **dog**, unde se observă multe exemple prezise ca alte animale, în acord cu valorile din classification report (recall scăzut).

Classification report (best CNN pe test)

Pe lângă metricile agregate și matricea de confuzie, am inclus și un **classification report** (precision/recall/F1 per clasă). Acesta oferă o vedere detaliată asupra performanței pe fiecare categorie și evidențiază clasele care contribuie cel mai mult la eroarea globală.

Classification report (best CNN on test) — pe clase

| Clasă    | Precision | Recall | F1-score | Support |
|----------|-----------|--------|----------|---------|
| airplane | 0.8366    | 0.8600 | 0.8481   | 500     |
| truck    | 0.6221    | 0.8660 | 0.7241   | 500     |
| bird     | 0.5841    | 0.6740 | 0.6258   | 500     |
| car      | 0.8636    | 0.7220 | 0.7865   | 500     |
| cat      | 0.5727    | 0.5040 | 0.5362   | 500     |
| deer     | 0.5073    | 0.7600 | 0.6085   | 500     |



| Clasă  | Precision | Recall | F1-score | Support |
|--------|-----------|--------|----------|---------|
| dog    | 0.6000    | 0.2220 | 0.3241   | 500     |
| horse  | 0.5857    | 0.7720 | 0.6661   | 500     |
| monkey | 0.6174    | 0.5260 | 0.5680   | 500     |
| ship   | 0.9494    | 0.6380 | 0.7632   | 500     |

Classification report (best CNN on test) — agregat

| Metrică      | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| Accuracy     |           |        | 0.6544   | 5000    |
| Macro avg    | 0.6739    | 0.6544 | 0.6451   | 5000    |
| Weighted avg | 0.6739    | 0.6544 | 0.6451   | 5000    |

Concluzii

- Clasele cu performanță ridicată sunt, în general, cele cu trăsături vizuale distinctive (ex. airplane, ship).
- Se observă confuzii între clase apropiate vizual/semantic (ex. cat vs dog, car vs truck), reflectate prin scăderi ale precision și/sau recall.
- dog are recall semnificativ mai mic decât majoritatea claselor, ceea ce indică multe exemple de dog clasificate ca alte animale (în acord cu matricea de confuzie).

Tabel rezumat rezultate (validare + test)

Pentru compararea configurațiilor am sintetizat, pentru fiecare model, performanța pe **validare** (Best Val Acc, Val Macro-F1) și pe **test** (Test Acc, Test Macro-F1), folosind aceeași configurație de antrenare (Adam, LR=1e-3, BATCH\_SIZE=128, EPOCHS=15, WEIGHT\_DECAY=1e-4). Modelul final este selectat prin **best checkpoint** pe baza val\_acc.

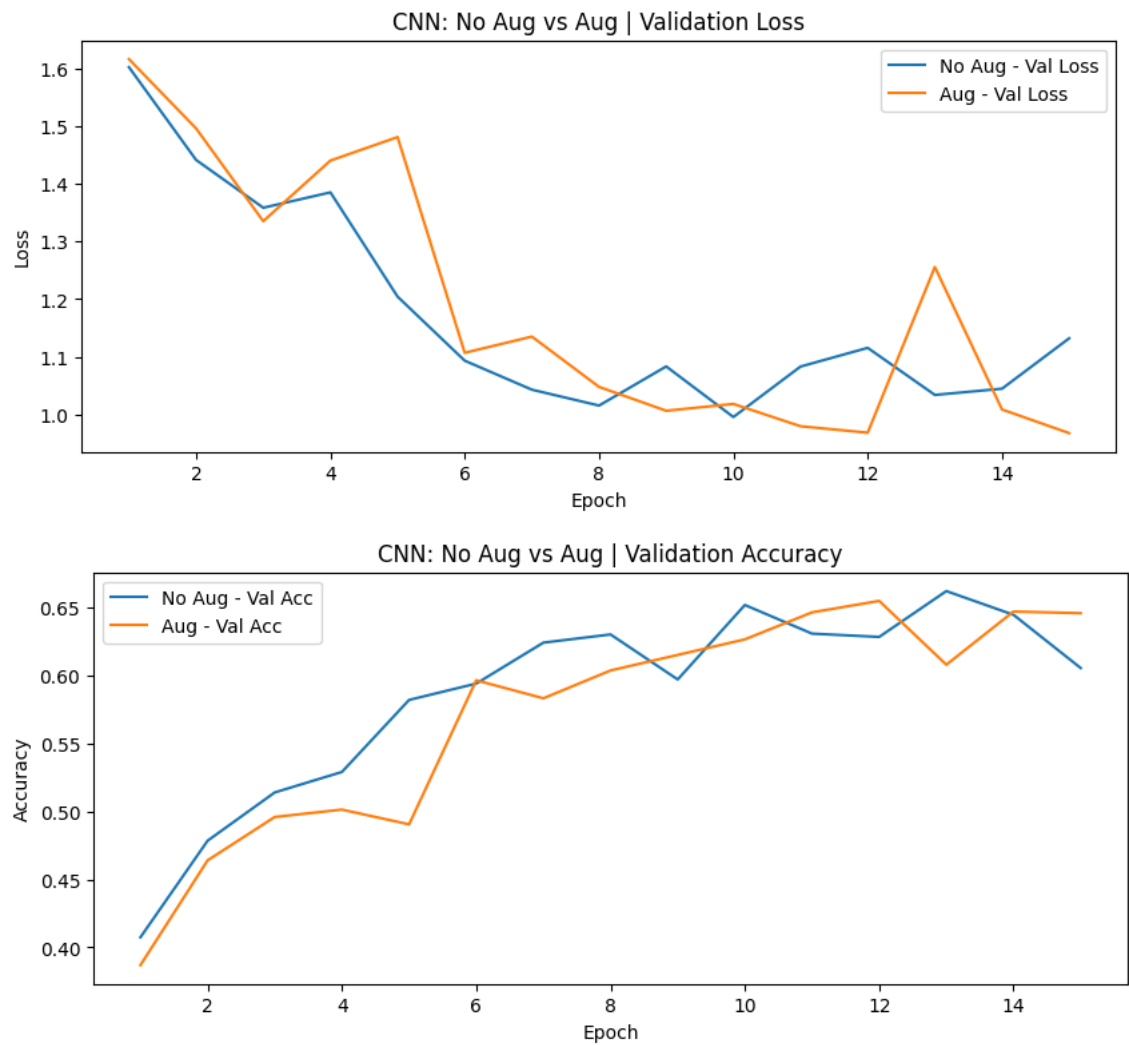
| Model         | Arhitectură (pe scurt)                                                                     | Aug (train) | Best Val Acc | Val Macro-F1 | Test Acc | Test Macro-F1 |
|---------------|--------------------------------------------------------------------------------------------|-------------|--------------|--------------|----------|---------------|
| MLP baseline  | Flatten → FC(1024) → ReLU → Dropout(0.3) → FC(512) → ReLU → Dropout(0.3) → FC(num_classes) | no          | 0.4075       | 0.3999       | 0.3978   | 0.3879        |
| CNN baseline  | Conv blocks (3→32→64→128→256) + BN + ReLU + MaxPool + FC head (Dropout 0.4)                | no          | 0.6617       | 0.6523       | 0.6544   | 0.6451        |
| CNN augmented | Aceeași arhitectură CNN; augmentări doar pe train                                          | yes         | 0.6544       | 0.6490       | 0.6528   | 0.6453        |

Observații:

- CNN depășește clar MLP atât pe validare cât și pe test, confirmând avantajul convoluțiilor pentru imagini (extragere de caracteristici spațiale).
- În această rulare, augmentarea nu crește vizibil Accuracy pe test față de CNN baseline (diferențe mici), dar menține un Macro-F1 comparabil, sugerând un efect de regularizare fără un câștig major de vârf.
- Conform criteriului best\_val\_acc, best CNN selectat este CNN baseline (are best\_val\_acc mai mare decât CNN aug).

Comparație directă augmentare

Pentru a evidenția efectul augmentării, am comparat direct evoluția pe validare între CNN baseline (no aug) și CNN augmented pe aceleași grafice:



Se observă că augmentarea tinde să ofere o evoluție mai stabilă a performanței pe validare (în special în zona finală), însă câștigul final pe test rămâne foarte apropiat de varianta fără augmentare în această configurație.

Salvarea checkpoint-ului (pentru fine-tuning pe Land patches)

După antrenarea modelului CNN pe **Imagebits**, am salvat starea modelului (`state_dict`) împreună cu metadate utile (dimensiunea imaginilor, numărul de clase și numele claselor) într-un fișier de tip checkpoint:

- `cnn_imagebits_64.pth`

Acest checkpoint va fi folosit ulterior ca **inițializare non-aleatoare** pentru experimentele de **fine-tuning** pe dataset-ul **Land patches**. La fine-tuning, voi păstra **aceeași rezoluție a imaginilor de intrare** ca în Imagebits (`IMG_SIZE = 96`), astfel încât arhitectura să poată încărca direct greutatele învățate.

Land Patches — modele, setup și experimente (baseline + augmentare + fine-tuning)

Dataset & split (support redus)

Setul **Land Patches** conține imagini RGB la rezoluția **64×64** și **10 clase** (AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, SeaLake). Spre deosebire de **Imagebits**, unde există un număr mare de exemple per clasă, aici **setul de antrenare are doar 200 imagini/clasă**, ceea ce crește semnificativ riscul de **overfitting** și face mai importante tehnici precum **augmentarea** și **fine-tuning-ul** (inițializare non-aleatoare).

Distribuție pe split-uri (balansată pe clase):

| Clasă                | Train | Val | Test |
|----------------------|-------|-----|------|
| AnnualCrop           | 200   | 100 | 700  |
| Forest               | 200   | 100 | 700  |
| HerbaceousVegetation | 200   | 100 | 700  |
| Highway              | 200   | 100 | 700  |

| Clasă         | Train | Val  | Test |
|---------------|-------|------|------|
| Industrial    | 200   | 100  | 700  |
| Pasture       | 200   | 100  | 700  |
| PermanentCrop | 200   | 100  | 700  |
| Residential   | 200   | 100  | 700  |
| River         | 200   | 100  | 700  |
| SeaLake       | 200   | 100  | 700  |
| Total         | 2000  | 1000 | 7000 |

Exemple vizuale:



Variabilitate intra-clasă (de ce e greu chiar și cu clase „similare”)

Deși clasele sunt definite semantic (tipuri de teren/peisaj), imagini din aceeași clasă pot arăta destul de diferit din cauza:

- **sezonalității** (diferențe de culoare/vegetation cover),
- **iluminării și condițiilor atmosferice**,
- **scării/perspectivei** și a texturilor locale,
- **fragmentării scenei** (patch-ul poate conține amestec de suprafețe).

Exemple tipice:

- **AnnualCrop / PermanentCrop / Pasture** pot varia mult ca textură și nuanță în funcție de stadiul vegetației.
- **River** poate apărea cu lățimi diferite, cu vegetație pe margini sau cu reflexii puternice.
- **Residential** poate include densități diferite ale construcțiilor (urban compact vs. suburban).

Această variabilitate intra-clasă este importantă deoarece, cu **doar 200 exemple/clasă**, modelul poate învăța corelații accidentale (de ex. anumite nuanțe sau fundaluri) în loc de tipare robuste.

Variabilitate inter-clasă (clase care se confundă natural)

Unele clase sunt vizual apropiate, mai ales la rezoluție mică (64×64), unde detaliile fine sunt limitate:

- **AnnualCrop vs PermanentCrop vs HerbaceousVegetation vs Pasture**: diferențe subtile de textură și pattern-uri (benzi/parcelare), ușor de confundat.
- **Highway vs Industrial vs Residential**: pot împărți structuri rectilinii și suprafețe artificiale; dacă patch-ul surprinde doar o porțiune mică, contextul devine insuficient.
- **River vs SeaLake**: ambele includ apă; diferențierea depinde de margini, context și culoare (care poate varia).

Comparativ cu **Imagebits**, unde obiectele (ex. **airplane, ship**) au adesea forme mai distinctive, aici separarea claselor se bazează mult pe **texturi și pattern-uri globale**. Asta favorizează CNN-uri care pot învăța filtre pentru texturi, dar poate cere și regularizare mai atentă.

Implicații pentru antrenare (comparat cu Imagebits)

- **Support redus la train** ⇒ risc mai mare de overfitting ⇒ augmentările (flip/rotație/jitter) sunt mai importante decât în Imagebits.
- **Rezoluție mică (64×64)** ⇒ modelul trebuie să se bazeze pe trăsături mai grosiere (texturi/pattern-uri), iar confuziile între clase apropiate sunt mai probabile.
- **Task de tip “scene/land cover classification”** (mai mult context + textură) ⇒ fine-tuning dintr-un model antrenat anterior poate ajuta la stabilizarea învățării, chiar dacă există un *domain gap* față de Imagebits.

Aceste observații motivează alegerea unui **CNN cu BatchNorm + pooling** (pentru stabilitate și reprezentări ierarhice) și testarea a două îmbunătățiri: **augmentare** (pentru generalizare) și **fine-tuning** (inițializare non-aleatoare).

Preprocesare & augmentări

Pentru a asigura o comparație corectă între modele și pentru a evita „leakage” (adică influențarea evaluării prin transformări aplicate și pe val/test), am separat explicit transformările în trei categorii: **eval (val/test)**, **train baseline** (fără augmentări) și **train augmentat** (augmentări doar pe train).

Transformări pentru evaluare (Val/Test) — fără augmentări

Pipeline-ul de evaluare este identic pentru **validare** și **test** și nu include augmentări:

- `Resize(IMG_SIZE×IMG_SIZE) → ToTensor → Normalize(mean, std)`

### De ce fără augmentări pe val/test?

Scopul validării și al testării este să măsoare performanța pe date „naturale” (nevăzute) într-un mod stabil și comparabil între experimente. Dacă am introduce augmentări pe val/test, am modifica distribuția datelor de evaluare și am amesteca efectul augmentărilor cu performanța reală a modelului.

### Train baseline (fără augmentări)

Experimentul baseline folosește exact aceleași transformări ca val/test:

- `Resize → ToTensor → Normalize`

Acest lucru este important pentru a avea un punct de referință „curat” atunci când evaluăm impactul arhitecturii (MLP vs CNN) sau al augmentării (CNN no-aug vs CNN aug).

---

### Normalizarea (Normalize) — rol și justificare

Am folosit `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`, valori standard folosite frecvent în pipeline-uri de viziune. Normalizarea:

- aduce canalele RGB la o scară comparabilă,
- stabilizează distribuția intrărilor către rețea,
- ajută optimizarea (gradient mai stabil, convergență mai rapidă), mai ales când există variații de lumină/contrast în imagini.

Chiar dacă aceste valori provin dintr-un set standard, ele funcționează bine ca normalizare generală; important este că se aplică **consistent** pe train/val/test.

---

### Train augmentat (augmentări potrivite pentru imagini satelitare)

Pentru a reduce overfitting-ul datorat suportului redus (200 imagini/clasă la train) și pentru a crește robustețea la variații realiste, am folosit un pipeline de augmentare aplicat **doar** pe train:

- `Resize → RandomHorizontalFlip(p=0.5) → RandomVerticalFlip(p=0.5) → RandomRotation(10°) → ColorJitter(brightness=0.15, contrast=0.15) → ToTensor → Normalize`

### Justificarea augmentărilor (problemă → alegere → efect așteptat)

- **Flip orizontal/vertical (p=0.5)**  
**Problemă:** în imagini satelitare, orientarea patch-urilor (N/S/E/W) poate varia, iar clasele (ex. Forest, River, SeaLake) nu depind de orientare.  
**Alegere:** flip H/V simulează schimbări de orientare fără a schimba semantica.  
**Efect așteptat:** crește invariance la orientare și reduce supraînvățarea unor pattern-uri specifice poziției.
- **Rotații mici ( $\pm 10^\circ$ )**  
**Problemă:** diferențe de aliniere între capturi / patch-uri pot apărea, iar modelul poate deveni sensibil la mici rotații.  
**Alegere:** rotațiile mici păstrează realismul (evitând rotații extreme care pot distorsiona puternic textura).  
**Efect așteptat:** îmbunătățește generalizarea și reduce dependența de alinierea exactă a texturilor.
- **ColorJitter (brightness/contrast = 0.15)**  
**Problemă:** iluminarea și contrastul pot varia în funcție de senzor, condiții atmosferice și sezon.  
**Alegere:** jitter moderat simulează aceste variații fără a altera clasa semantică (ex. o pădure rămâne pădure).  
**Efect așteptat:** robustețe mai mare la schimbări de lumină/contrast și val performance mai stabilă.

---

### DataLoaders & setări (implementare)

Am construit DataLoader-e separate pentru:

- `train_loader_base` (train fără aug),
- `train_loader_aug` (train cu aug),
- `val_loader` și `test_loader` (fără aug, `shuffle=False`).

Setările importante:

- `shuffle=True` doar pe train (pentru amestecarea exemplurilor),
- `shuffle=False` pe val/test (pentru evaluare deterministă),
- batch size și număr de workeri controlați prin `CFG`.

Această separare permite o comparație directă și corectă între **CNN no-aug** și **CNN aug**, unde singura diferență reală este prezența augmentărilor în timpul antrenării.

### Setup & configurație (CFG)

Pentru a menține experimentele reproductibile și comparabile, am centralizat hiperparametrii într-o configurație `CFG`, utilizată constant în toate rulările pe **Land Patches**.

**Reproductibilitate (seed):** am fixat `seed=42` pentru Python / NumPy / PyTorch (și CUDA, dacă este disponibil), astfel încât rezultatele să fie cât mai stabile între rulări.

### Configurația experimentelor (CFG):

- Director date: `DATA_DIR = "land_patches"` (structură `train/, val/, test/` per clasă)
- Rezoluție imagini: `IMG_SIZE = 64`
- Batch size: `BATCH_SIZE = 64`

- Epoci: `EPOCHS = 20`
- Optimizator: **Adam**
- Learning rate: `LR = 1e-3`
- Regularizare: `WEIGHT_DECAY = 1e-4`
- Device: `DEVICE = "cuda"` dacă este disponibil, altfel `"cpu"`
- Data loading: `NUM_WORKERS = 2`

#### Justificare (de ce aceste alegeri):

- `IMG_SIZE=64` este rezoluția nativă a dataset-ului, deci evită distorsiuni inutile și reduce costul de antrenare.
- `BATCH_SIZE=64` este un compromis între stabilitatea gradientului și consumul de memorie (mai ales pe GPU).
- `EPOCHS=20` permite observarea clară a convergenței și a eventualului overfitting (vizibil în curbele train/val).
- `LR=1e-3` pentru Adam este o valoare standard care oferă de obicei convergență stabilă; pentru fine-tuning (unde e util un pas mai mic) voi folosi un LR redus.
- `WEIGHT_DECAY=1e-4` adaugă regularizare L2 pentru a reduce supraînvățarea, mai importantă aici decât în Imagebits din cauza suportului redus (200 imagini/clasă la train).
- `NUM_WORKERS=2` accelerează încărcarea datelor fără a complica rularea (mai ales pe medii limitate).

#### Modele utilizate (MLP vs CNN)

Pentru a evalua impactul arhitecturii asupra clasificării pe **Land Patches (64×64)**, am implementat două modele: un **MLP** (baseline) și un **CNN** (convoluțional). Alegerea celor două familii urmărește să compare un model care tratează imaginea ca un vector (MLP) cu un model care exploatează explicit structura spațială (CNN).

#### MLP baseline (SimpleMLP, 64×64)

##### Arhitectură:

- Intrare: imagine RGB  $3 \times 64 \times 64$ , aplatizată într-un vector de dimensiune  $3 \cdot 64 \cdot 64 = 12288$
- Rețea:
  - `Flatten → Linear(12288, 512) → ReLU → Dropout(0.3)`
  - `Linear(512, 128) → ReLU → Dropout(0.2)`
  - `Linear(128, 10)` (logits pentru cele 10 clase)

**Cum funcționează (intuitiv):** MLP-ul învață **combinații globale** ale pixelilor: fiecare neuron dintr-un strat fully-connected este conectat la toate valorile de intrare. Astfel, modelul poate învăța corelații utile, dar **nu are un mecanism explicit** pentru a detecta tipare locale (muchii, texturi) sau pentru a folosi relațiile de vecinătate dintre pixeli.

**Justificare (de ce îl folosim):** MLP servește ca **baseline simplu**, util pentru a cuantifica cât câștigăm atunci când trecem la o arhitectură specializată pentru imagini (CNN). Pentru Land Patches, unde clasele depind în mare parte de **texturi/pattern-uri** (agricultură, vegetație, zone urbane), ne așteptăm ca MLP-ul să fie limitat tocmai pentru că pierde structura spațială prin aplatizare.

#### CNN simplu (SimpleCNN, 64×64)

**Arhitectură (conform implementării):** CNN-ul este format dintr-un extractor de caracteristici (**features**) urmat de un clasificator (**classifier**):

**Extractor (features):** 3 blocuri de forma `Conv → BatchNorm → ReLU → MaxPool`

1. `Conv2d(3→32, k=3, padding=1) → BN(32) → ReLU → MaxPool(2)`  
Rezoluție:  $64 \times 64 \rightarrow 32 \times 32$
2. `Conv2d(32→64, k=3, padding=1) → BN(64) → ReLU → MaxPool(2)`  
Rezoluție:  $32 \times 32 \rightarrow 16 \times 16$
3. `Conv2d(64→128, k=3, padding=1) → BN(128) → ReLU → MaxPool(2)`  
Rezoluție:  $16 \times 16 \rightarrow 8 \times 8$

La final, tensorul are dimensiunea  $128 \times 8 \times 8$  și este trecut către head-ul de clasificare.

##### Clasificator (classifier):

- `Flatten → Linear(128·8·8, 256) → ReLU → Dropout(0.4) → Linear(256, 10)`

##### Cum funcționează (intuitiv):

- **Convoluțiile** învață filtre locale care detectează tipare vizuale (inițial muchii/texturi, apoi combinații mai complexe).
- **Pooling-ul** reduce treptat rezoluția spațială, păstrând informația importantă și crescând invariance la mici translații.
- **BatchNorm** stabilizează activările și ajută antrenarea să fie mai robustă (gradient mai stabil, convergență mai consistentă).
- Clasificatorul final folosește caracteristicile extrase pentru a decide clasa.

##### Justificare (problemă → alegere → efect așteptat):

- **Problemă:** Land Patches este un task bazat pe **texturi/pattern-uri** și are suport redus la train; un model care nu păstrează structura spațială poate generaliza slab.
- **Alegere:** CNN cu convoluții + pooling pentru a extrage trăsături spațiale și **BatchNorm** pentru stabilitate.
- **Efect așteptat:** performanță semnificativ mai bună decât MLP (în special pe clase apropiate), plus curbe train/val mai stabile.

## Regularizare folosită în modele

- **MLP:** Dropout(0.3) și Dropout(0.2) pentru a reduce supraînvățarea în straturile dense.
- **CNN:** Dropout(0.4) în head, + BatchNorm în extractor; împreună ajută la generalizare într-un scenariu cu train redus.

---

## Setup de antrenare (comun)

- Loss: **CrossEntropyLoss**
- Optimizator: **Adam**
- Hiperparametri: `LR = 1e-3`, `WEIGHT_DECAY = 1e-4`
- Antrenare: `BATCH_SIZE = 64`, `EPOCHS = 20`
- Scheduler: **none** (learning rate constant)
- Regularizare: **weight decay** + **dropout** (în MLP/CNN) + **BatchNorm** (în CNN) + **augmentare** (doar în experimentele cu aug)
- Reproducibilitate: `SEED = 42` (fixat pentru Python/NumPy/PyTorch; CUDA dacă e disponibil)
- Device: `cuda` dacă este disponibil, altfel `cpu`
- Selecție model: **best checkpoint** după `val_acc` (se salvează și se încarcă la final starea cu cel mai bun `val_acc`)
- Metrici raportate: **Accuracy** și **Macro-F1** (Macro-F1 e relevant pentru evaluare echilibrată pe clase)

**Notă despre evaluare:** pe `val/test` nu se aplică augmentări, pentru a păstra o evaluare stabilă și comparabilă între configurații. Am ales Adam (`LR=1e-3`) pentru convergență stabilă pe setul de antrenare mic, iar regularizarea (weight decay + dropout + augmentare) a fost folosită pentru a reduce overfitting-ul observat prin diferența train–val.

---

## Experimente (Land Patches)

### Exp. 1 — MLP baseline (no aug)

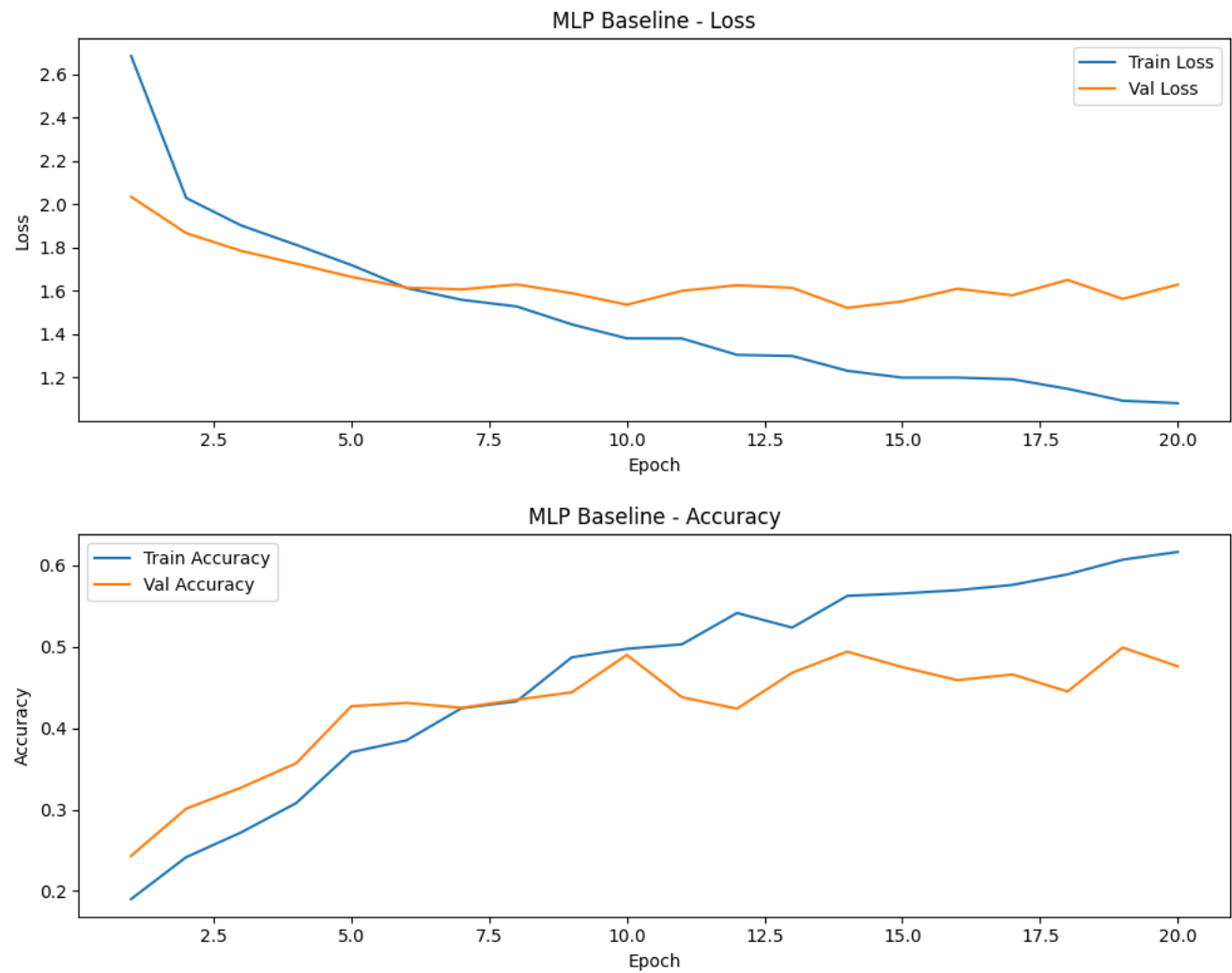
**Scopul experimentului.** Acest experiment oferă un **baseline** pentru Land Patches, folosind un MLP care primește imaginea aplatizată (fără structură spațială). Pentru un task de tip *land cover classification* (bazat pe texturi și pattern-uri), ne așteptăm ca MLP-ul să fie limitat, mai ales când setul de antrenare este mic (200 imagini/clasă).

#### Setup (rezumat):

- Model: `SimpleMLP` (Flatten → FC(512) → ReLU → Dropout(0.3) → FC(128) → ReLU → Dropout(0.2) → FC(10))
- Loss: **CrossEntropyLoss**
- Optimizator: **Adam** (`LR=1e-3`, `weight_decay=1e-4`)
- `BATCH_SIZE=64`, `EPOCHS=20`
- Date: train **fără augmentări**, val/test **fără augmentări**
- Selecție: *best checkpoint* după `val_acc` (în practică, raportăm și max(`val_acc`) pentru comparație)

---

## Curbe train/val (interpretare)



Ce se observă în curbe:

- **Train loss** scade constant de la început până la final, ceea ce arată că modelul reușește să se potrivească (fit) pe datele de antrenare.
- **Val loss** scade în primele epoci, apoi tinde să se **stabilizeze sau să crească ușor** spre final.  
Acesta este un semnal tipic de **overfitting**: modelul continuă să învețe detalii specifice setului mic de train, dar nu mai îmbunătățește generalizarea.
- **Train accuracy** crește până în jur de ~0.6, în timp ce **val accuracy** se plafonează în jur de ~0.45–0.50 și fluctuează.  
Diferența train–val (gap) confirmă încă o dată overfitting-ul și limitele MLP-ului pe acest tip de date.

Interpretare (problemă → concluzie):

MLP-ul nu are inductive bias pentru imagini (nu vede vecinătăți, texturi locale), deci are dificultăți în a învăța pattern-uri robuste de teren. Cu suport redus, acest lucru se traduce prin **generalizare modestă** și confuzii între clase vizual similare.

Rezultate pe test (Accuracy & Macro-F1)

Mai jos sunt metricile finale pe setul de **test** (700 imagini/clasă), unde raportăm atât **Accuracy** cât și **Macro-F1** (Macro-F1 evidențiază dacă există clase cu performanță mult mai slabă).

Tabel rezultate (MLP, test):

| Model        | Aug (train) | Test Accuracy | Test Macro-F1 |
|--------------|-------------|---------------|---------------|
| MLP baseline | no          | 0.4573        | 0.4371        |

Classification report (MLP baseline, test)

Pe clase

| Clasă                | Precision | Recall | F1-score | Support |
|----------------------|-----------|--------|----------|---------|
| AnnualCrop           | 0.4086    | 0.5814 | 0.4800   | 700     |
| Forest               | 0.5364    | 0.6414 | 0.5843   | 700     |
| HerbaceousVegetation | 0.3366    | 0.2457 | 0.2841   | 700     |

| Clasă         | Precision | Recall | F1-score | Support |
|---------------|-----------|--------|----------|---------|
| Highway       | 0.2300    | 0.1557 | 0.1857   | 700     |
| Industrial    | 0.7579    | 0.4786 | 0.5867   | 700     |
| Pasture       | 0.6153    | 0.7243 | 0.6654   | 700     |
| PermanentCrop | 0.3194    | 0.1414 | 0.1960   | 700     |
| Residential   | 0.3845    | 0.4757 | 0.4253   | 700     |
| River         | 0.4070    | 0.3343 | 0.3671   | 700     |
| SeaLake       | 0.4773    | 0.7943 | 0.5962   | 700     |

Agregat

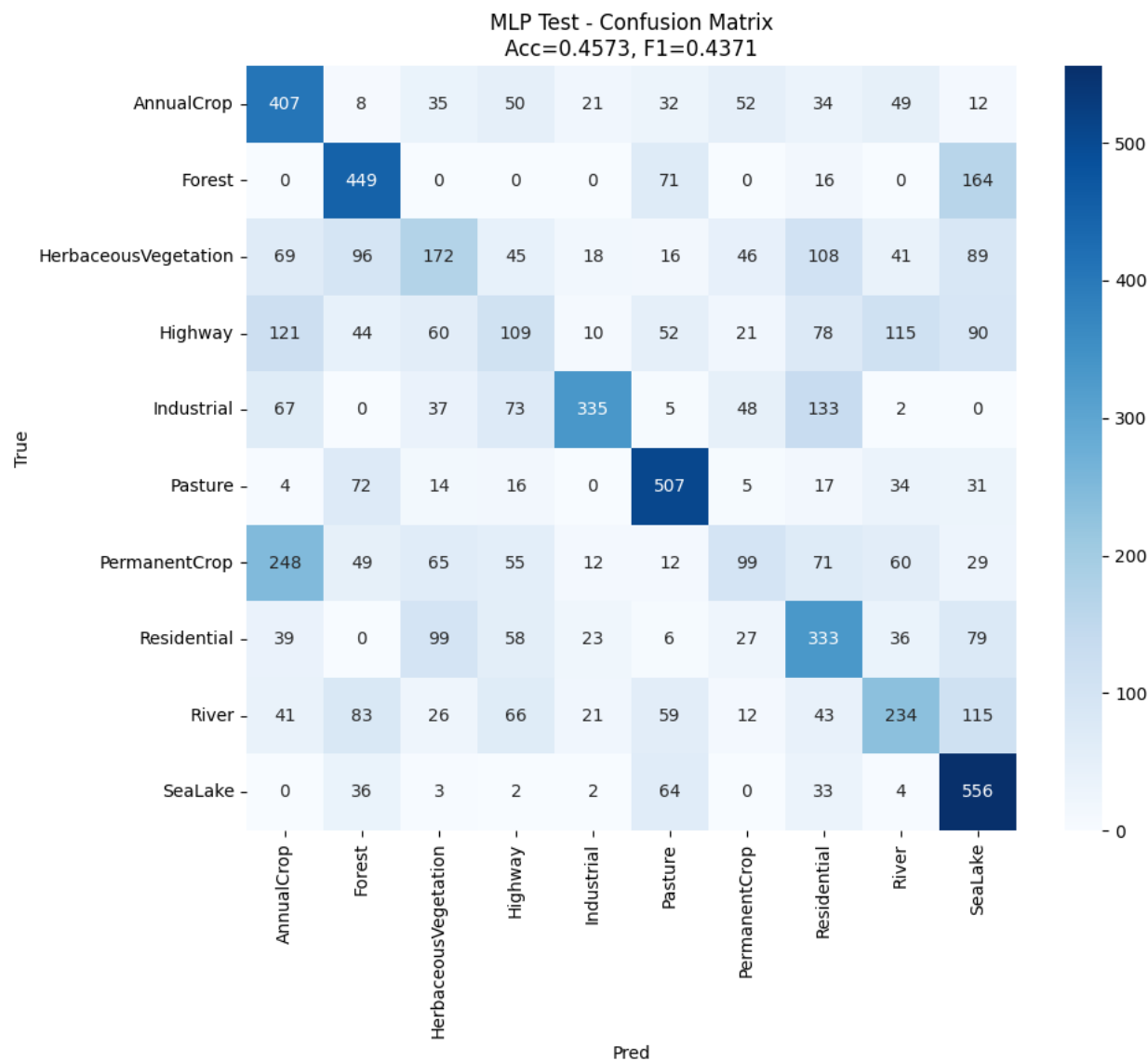
| Metrică      | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| Accuracy     |           |        | 0.4573   | 7000    |
| Macro avg    | 0.4473    | 0.4573 | 0.4371   | 7000    |
| Weighted avg | 0.4473    | 0.4573 | 0.4371   | 7000    |

Comentarii / interpretare:

- Performanța este **neuniformă între clase**, ceea ce se reflectă în **Macro-F1 = 0.4371**.
- SeaLake** (recall 0.7943) și **Pasture** (recall 0.7243) sunt cele mai bine recunoscute, probabil datorită unor pattern-uri mai distinctive.
- Highway** (recall 0.1557) și **PermanentCrop** (recall 0.1414) sunt cele mai problematice, ceea ce sugerează confuzii frecvente între clase cu texturi/pattern-uri similare la rezoluție 64×64.
- Industrial** are **precision mare** (0.7579) dar **recall moderat** (0.4786): când modelul prezice „Industrial” tinde să aibă dreptate, dar ratează o parte semnificativă din exemplele reale.

Matricea de confuzie (MLP, test)





**Cum se interpretează:**  
Diagonala (valorile cele mai mari) reprezintă clasificările corecte; celulele cu valori mari în afara diagonalei indică **confuzii sistematice** între clase.

- Confuzii importante (vizibile în matrice):**
- **PermanentCrop** este confundată frecvent cu **AnnualCrop** (valoare mare pe rândul PermanentCrop, coloana AnnualCrop), ceea ce e de așteptat deoarece ambele clase pot avea pattern-uri agricole similare.
  - **Highway** se împrăștie către mai multe clase (ex. **AnnualCrop**, **River**, **SeaLake**), sugerând că MLP-ul nu surprinde bine structuri liniare la rezoluție mică.
  - **HerbaceousVegetation** are multe predicții în alte clase (ex. **Forest**, **Residential**), reflectând faptul că diferențele de textură sunt subtile și MLP-ul nu le captează bine.

Concluzie (Exp. 1)

MLP-ul oferă un baseline util, dar rezultatele (Accuracy ~0.46, Macro-F1 ~0.44) și tiparele din matricea de confuzie indică limitările unui model fără structură spațială pentru un task dominat de **texturi/pattern-uri**. Aceste observații motivează trecerea la un **CNN** (convoluții + pooling + BatchNorm) și explorarea de **augmentări** pentru reducerea overfitting-ului pe suport redus.

Exp. 2 — CNN baseline (no aug)

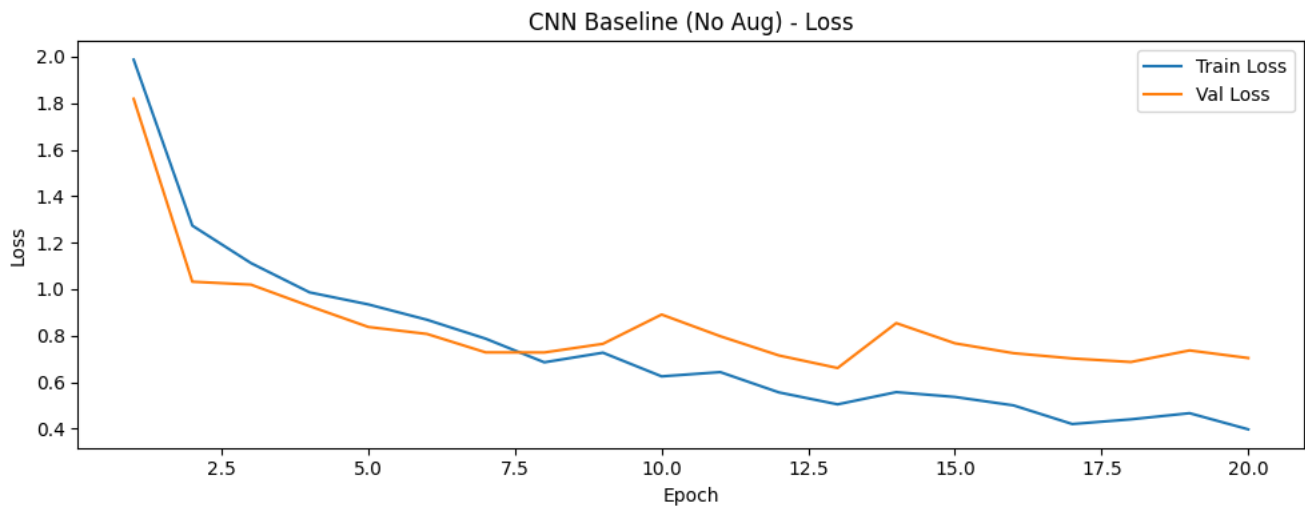
**Scopul experimentului.** După baseline-ul MLP, trecem la un **CNN** pentru a introduce *inductive bias* specific imaginilor: convoluțiile pot învăța filtre locale (texturi/pattern-uri), iar **MaxPool** și **BatchNorm** cresc robustețea și stabilizează antrenarea. Pe Land Patches (clasificare de scene/teren), ne așteptăm ca un CNN să capteze mult mai bine structurile și texturile decât un MLP care aplatizează imaginea.

Setup (rezumat):

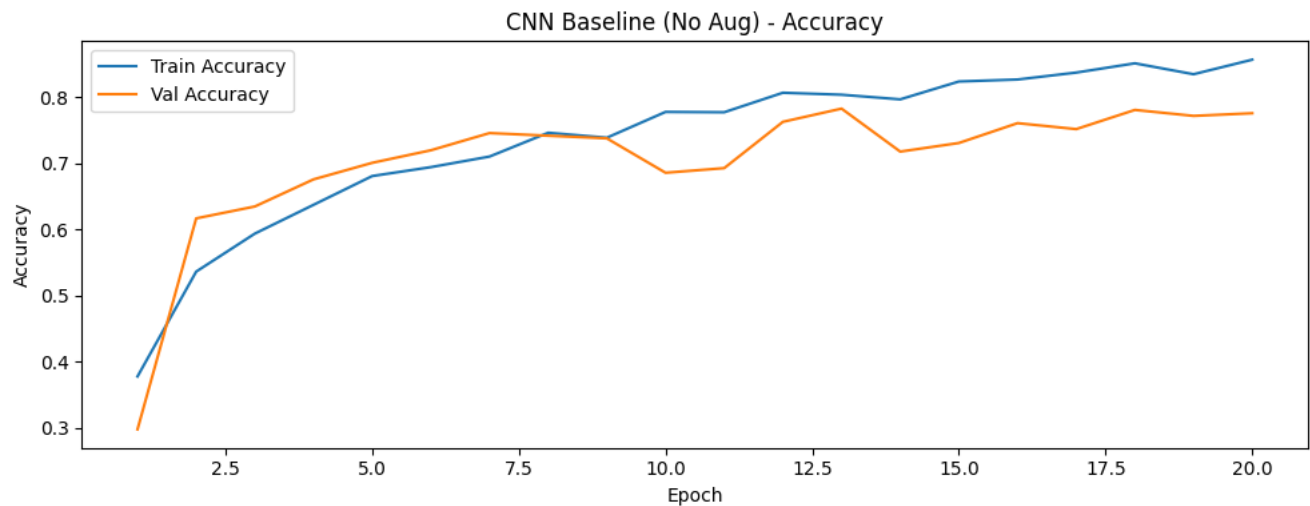
- Model: **SimpleCNN** (Conv→BN→ReLU→Pool x3; head: FC(256)+Dropout(0.4))
- Loss: **CrossEntropyLoss**
- Optimizator: **Adam** (LR=1e-3, weight\_decay=1e-4)
- **BATCH\_SIZE=64**, **EPOCHS=20**
- Date: train **fără augmentări**, val/test **fără augmentări**

- Selecție: *best checkpoint* după `val_acc` (raportăm `max(val_acc)` și evaluarea finală pe test)

Curbe train/val (interpretare)



Figuri:



Ce se observă în curbe:

- În primele epoci, atât **train loss** cât și **val loss** scad rapid, indicând că modelul învață eficient reprezentări utile.
- **Val accuracy** crește puternic în primele epoci și se stabilizează în jurul unui platou ridicat (~0.75–0.78), mult peste nivelul MLP.
- Spre final, se vede un **gap moderat train-val** (train accuracy continuă să urce ușor, în timp ce val accuracy oscilează). Acest comportament sugerează început de **overfitting**, însă semnificativ mai redus decât în cazul MLP, datorită convoluțiilor + BatchNorm + dropout în head.

**Interpretare (de ce CNN ajută):** CNN-ul poate recunoaște pattern-uri locale (texturi de vegetație, structuri artificiale, margini apă-uscă), ceea ce explică saltul clar de performanță față de MLP, mai ales pe clase vizual apropiate.

Rezultate pe test (Accuracy & Macro-F1)

Tabel rezultate (CNN baseline, test):

| Model        | Aug (train) | Test Accuracy | Test Macro-F1 |
|--------------|-------------|---------------|---------------|
| CNN baseline | no          | 0.7689        | 0.7656        |

Observație comparativă față de MLP:

CNN baseline îmbunătățește semnificativ performanța față de MLP (care avea ~0.4573 acc și ~0.4371 macro-F1), confirmând că exploatarea structurii spațiale este critică pentru acest dataset.

Classification report (CNN baseline, test)

Pe clase

| Clasă      | Precision | Recall | F1-score | Support |
|------------|-----------|--------|----------|---------|
| AnnualCrop | 0.8351    | 0.7886 | 0.8112   | 700     |

| Clasă                | Precision | Recall | F1-score | Support |
|----------------------|-----------|--------|----------|---------|
| Forest               | 0.7907    | 0.9714 | 0.8718   | 700     |
| HerbaceousVegetation | 0.5783    | 0.8757 | 0.6966   | 700     |
| Highway              | 0.6778    | 0.5529 | 0.6090   | 700     |
| Industrial           | 0.9087    | 0.8100 | 0.8565   | 700     |
| Pasture              | 0.8606    | 0.6700 | 0.7534   | 700     |
| PermanentCrop        | 0.7328    | 0.5329 | 0.6170   | 700     |
| Residential          | 0.7898    | 0.9557 | 0.8649   | 700     |
| River                | 0.7143    | 0.6857 | 0.6997   | 700     |
| SeaLake              | 0.9094    | 0.8457 | 0.8764   | 700     |

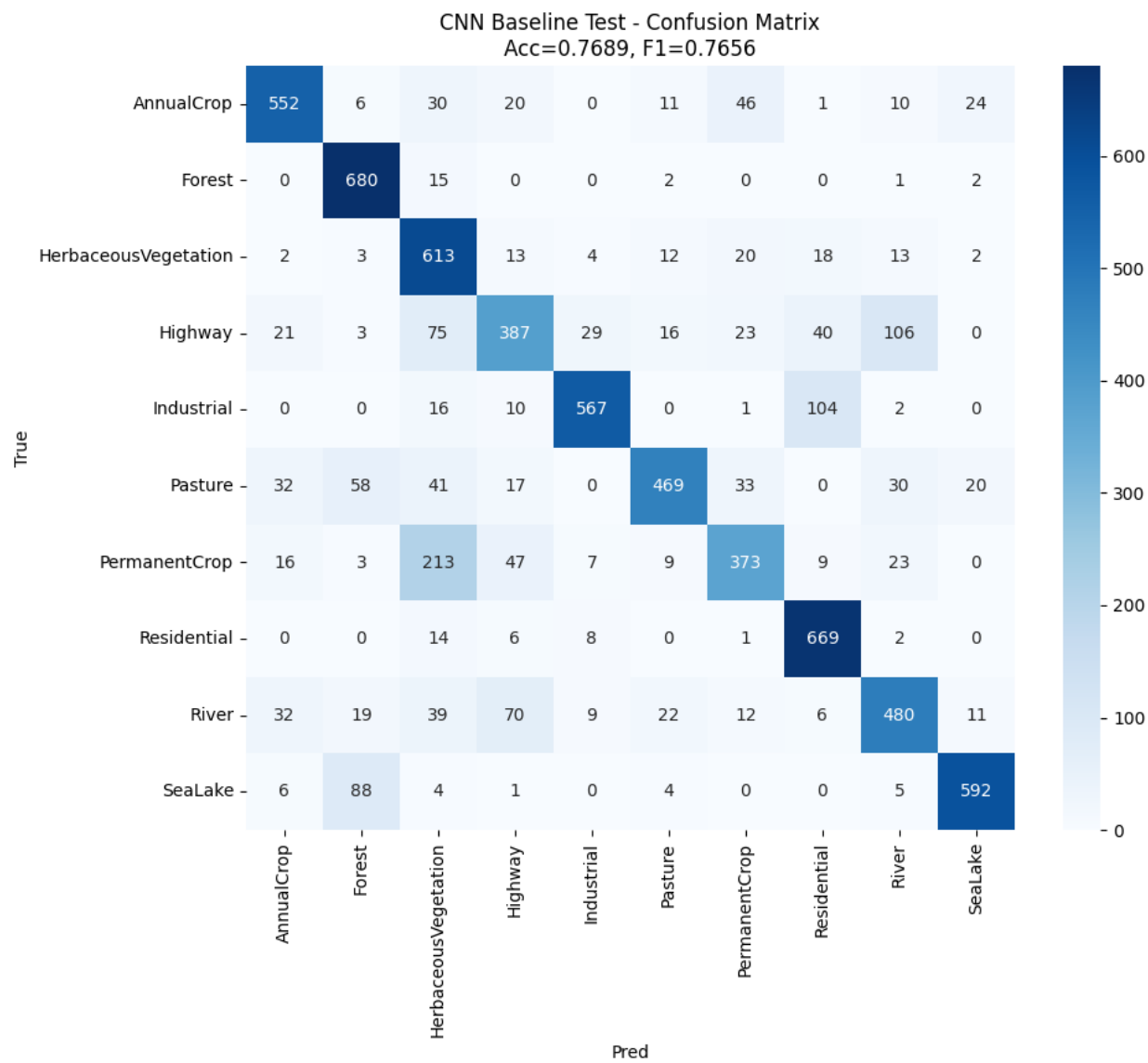
Agregat

| Metrică      | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| Accuracy     |           |        | 0.7689   | 7000    |
| Macro avg    | 0.7797    | 0.7689 | 0.7656   | 7000    |
| Weighted avg | 0.7797    | 0.7689 | 0.7656   | 7000    |

Comentarii / interpretare:

- Clase foarte bine recunoscute (recall ridicat): **Forest** (0.9714) și **Residential** (0.9557), urmate de **SeaLake** (0.8457) și **Industrial** (0.8100). Aceste clase au pattern-uri relativ distinctive (texturi dense de vegetație, structuri urbane, suprafețe de apă, zone industriale).
- Clase mai dificile: **Highway** (recall 0.5529) și **PermanentCrop** (recall 0.5329). Pentru aceste clase, contextul la 64×64 poate fi insuficient sau pattern-urile pot fi similare cu alte tipuri de teren (ex. agricultură vs vegetație, infrastructură vs urban).
- **HerbaceousVegetation** are recall mare (0.8757) dar precision mai mică (0.5783), ceea ce sugerează că modelul prezice această clasă relativ des, incluzând și exemple care aparțin altor clase de vegetație/agricultură.

Matricea de confuzie (CNN baseline, test)



Ce evidențiază matricea:

- Diagonala este puternică pentru clase ca Forest, Residential, SeaLake, confirmând performanța bună.
- Confuzia rămâne vizibilă între clase apropiate vizual:
  - PermanentCrop este confundată frecvent cu HerbaceousVegetation (și parțial cu AnnualCrop), ceea ce reflectă pattern-uri agricole/vegetale similare la rezoluție mică.
  - Highway are confuzii cu River și clase construite/vegetale în funcție de contextul patch-ului (structuri liniare vs margini).

Concluzie (Exp. 2)

CNN baseline obține un salt major față de MLP și reprezintă un punct de referință solid pentru Land Patches. Curbele indică o convergență bună și doar un overfitting moderat spre final, ceea ce motivează în experimentul următor testarea augmentărilor pentru a îmbunătăți generalizarea.

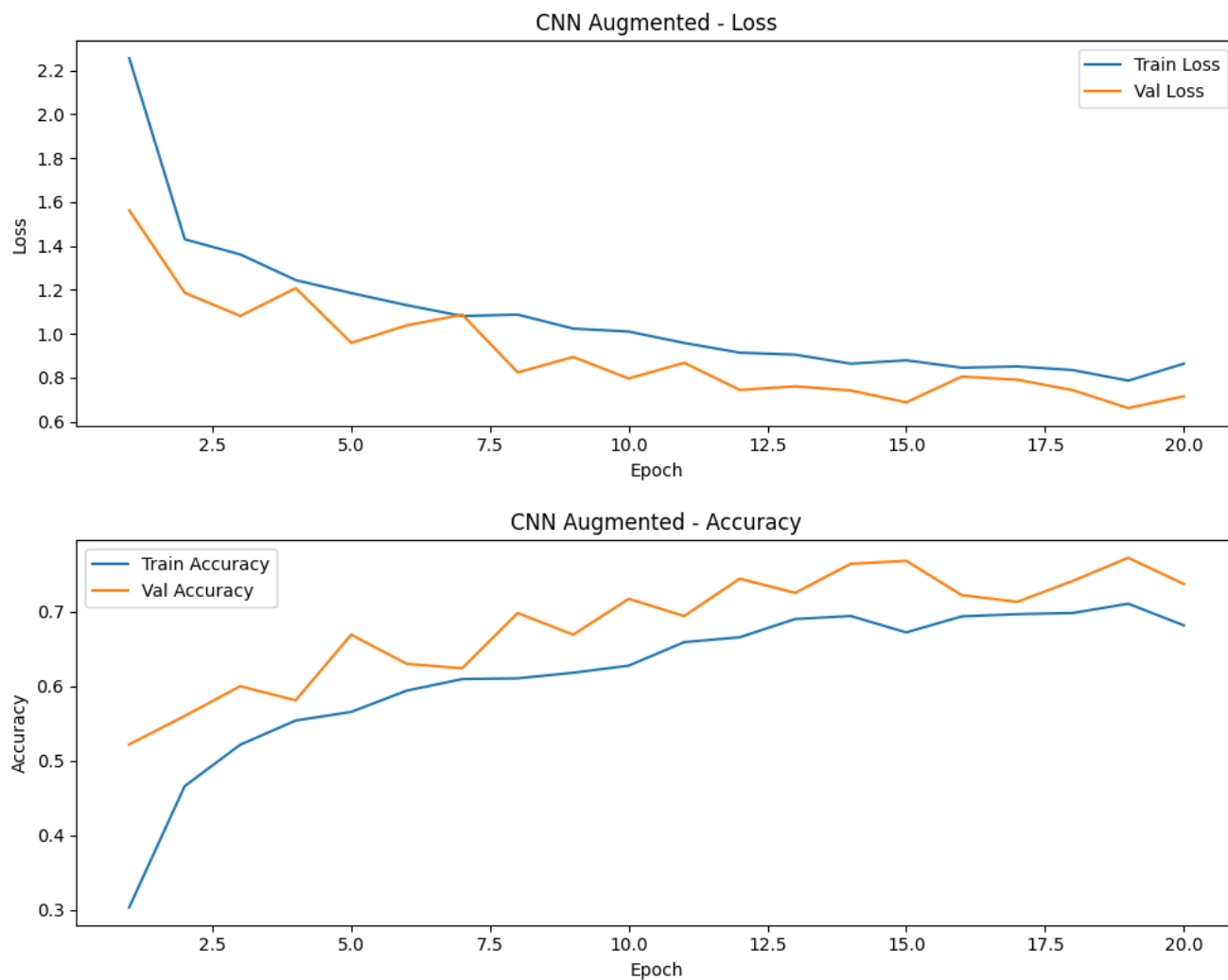
Exp. 3 — CNN + augmentare (train aug)

**Scopul experimentului.** După CNN baseline (fără augmentări), introducem augmentări **doar pe train** pentru a combate **overfitting-ul** produs de suportul redus (200 imagini/clasă la train). În mod ideal, augmentările ar trebui să crească robustețea și să îmbunătățească generalizarea pe val/test.

Setup (rezumat):

- Model: **același SimpleCNN** ca în Exp. 2 (diferența este DOAR pipeline-ul de train)
- Loss: **CrossEntropyLoss**
- Optimizator: **Adam** (LR=1e-3, weight\_decay=1e-4)
- BATCH\_SIZE=64, EPOCHS=20
- Date: train\_loader\_aug (cu aug), val/test fără aug
- Selecție: best checkpoint după val\_acc (raportăm și max(val\_acc) + rezultate pe test)

Curbe train/val (interpretare)



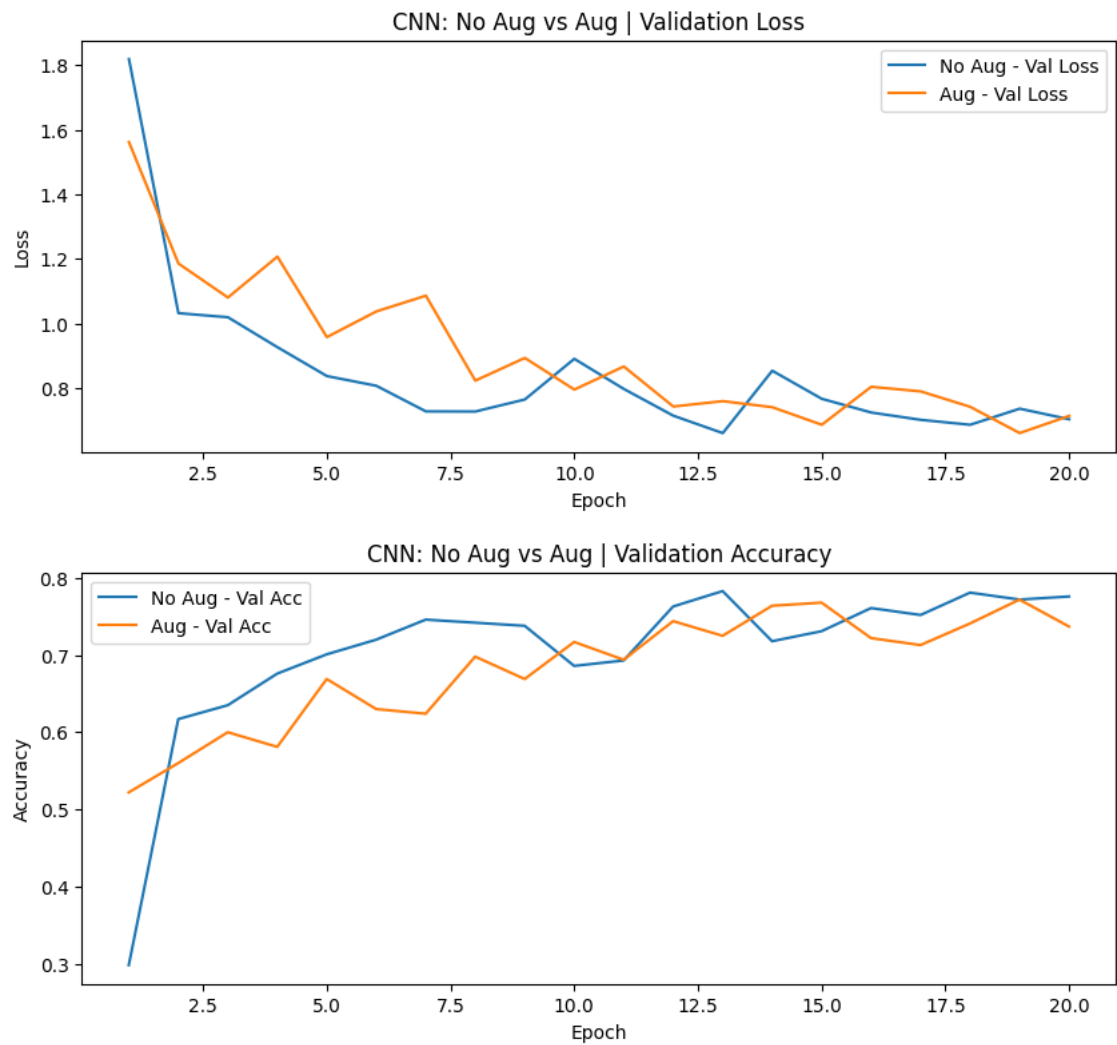
#### Ce se observă în curbe:

- **Train accuracy** crește mai lent decât la CNN baseline, ceea ce este normal: augmentările fac sarcina de învățare „mai grea” și acționează ca regularizare.
- **Val accuracy** este relativ competitivă și, în unele epoci, se apropie de CNN baseline, însă nu oferă un salt clar peste acesta.
- **Gap-ul train-val** tinde să fie mai mic decât în experimentul fără augmentări, sugerând un efect de regularizare (modelul nu se mai potrivește excesiv pe train).

**Interpretare:** augmentările reduc supraînvățarea, dar pot induce și un ușor **underfitting** (sau o distribuție de train prea „diferită”), mai ales dacă sunt relativ agresive pentru un dataset mic și pentru un model simplu.

---

#### Comparație directă (no aug vs aug)



Ce arată comparația:

- Curbele confirmă că augmentarea schimbă dinamica de antrenare: în general, val loss/val acc sunt **mai stabile** sau cu fluctuații diferite față de varianta fără aug.
- În această rulare, nu se vede un avantaj constant al augmentării pe validare (curbele se intersectează și diferențele sunt mici), ceea ce explică de ce și performanța pe test rămâne sub baseline.

Rezultate pe test (Accuracy & Macro-F1)

Tabel rezultate (CNN augmented, test):

| Model         | Aug (train) | Test Accuracy | Test Macro-F1 |
|---------------|-------------|---------------|---------------|
| CNN augmented | yes         | 0.7319        | 0.7307        |

Observație importantă:

Deși augmentarea a fost introdusă pentru a îmbunătăți generalizarea, în această configurație **CNN baseline (no aug)** rămâne mai bun pe test (0.7689 vs 0.7319). O explicație plauzibilă este că augmentările (în special combinația flip H/V + rotație + jitter) pot fi **prea agresive** pentru unele clase sau pot introduce exemple mai puțin realiste pentru distribuția de test, ceea ce reduce performanța finală (mai ales fără tuning suplimentar al LR/epochs).

Classification report (CNN augmented, test)

Pe clase

| Clasă                | Precision | Recall | F1-score | Support |
|----------------------|-----------|--------|----------|---------|
| AnnualCrop           | 0.8644    | 0.7743 | 0.8169   | 700     |
| Forest               | 0.7177    | 0.8571 | 0.7812   | 700     |
| HerbaceousVegetation | 0.5827    | 0.7400 | 0.6520   | 700     |
| Highway              | 0.4875    | 0.5314 | 0.5085   | 700     |
| Industrial           | 0.8841    | 0.9157 | 0.8996   | 700     |
| Pasture              | 0.7213    | 0.7800 | 0.7495   | 700     |

| Clasă         | Precision | Recall | F1-score | Support |
|---------------|-----------|--------|----------|---------|
| PermanentCrop | 0.6271    | 0.6414 | 0.6342   | 700     |
| Residential   | 0.8462    | 0.9671 | 0.9027   | 700     |
| River         | 0.7862    | 0.5200 | 0.6260   | 700     |
| SeaLake       | 0.9764    | 0.5914 | 0.7367   | 700     |

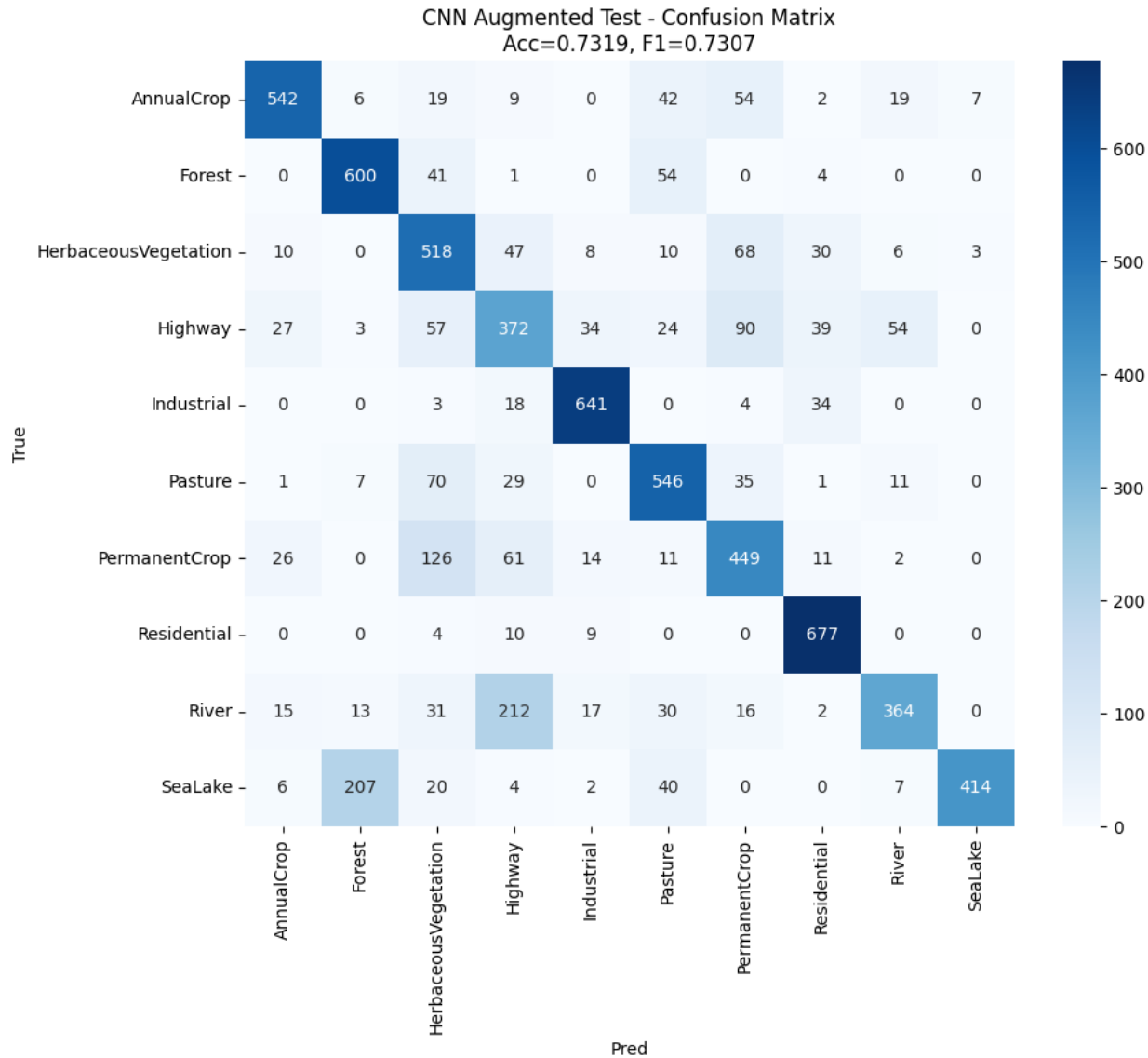
Agregat

| Metrică      | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| Accuracy     |           |        | 0.7319   | 7000    |
| Macro avg    | 0.7494    | 0.7319 | 0.7307   | 7000    |
| Weighted avg | 0.7494    | 0.7319 | 0.7307   | 7000    |

Comentarii / interpretare:

- **Residential** și **Industrial** rămân foarte bune (recall ~0.97 și ~0.92), ceea ce sugerează că pattern-urile urbane/industriale sunt robuste chiar și cu augmentare.
- Scăderi notabile apar la **River** (recall 0.52) și **SeaLake** (recall 0.59). Asta sugerează confuzii pe clasele de apă, posibil amplificate de variațiile de lumină/contrast (ColorJitter) și de schimbări de orientare.
- **Highway** rămâne una dintre cele mai dificile clase (recall 0.53), ceea ce este coerent cu faptul că structurile liniare pot fi greu de surprins la 64×64, iar rotațiile/flip-urile pot modifica contextul local.

Matricea de confuzie (CNN augmented, test)



Confuzii importante (vizibile în matrice):

- `River` → `Highway` este o confuzie majoră (multe exemple de River prezise ca Highway), ceea ce poate apărea când patch-urile conțin structuri liniare sau margini înguste care seamănă (râu îngust vs drum).
  - `SeaLake` → `Forest` apare frecvent, indicând că unele scene de apă pot avea texturi/culori care se apropie de vegetație întunecată, mai ales sub jitter de lumină/contrast.
  - `PermanentCrop` ↔ `HerbaceousVegetation` rămâne o pereche dificilă (pattern-uri agricole/vegetale similare), iar augmentarea nu elimină complet această ambiguitate.
- 

### Concluzie (Exp. 3)

Augmentarea reduce parțial tendința de overfitting (dinamică mai regularizată în curbe), însă în această rulare nu depășește CNN baseline pe test. Pentru a transforma augmentarea într-un câștig consistent, ar putea fi necesară ajustarea intensității augmentărilor (mai puțin jitter sau rotații), un LR mai mic / scheduler, sau antrenare mai lungă pentru a compensa dificultatea suplimentară introdusă de augmentări.

---

### Exp. 4 — Fine-tuning Imagebits → Land Patches (inițializare non-aleatoare)

**Scopul experimentului.** Land Patches are suport redus la antrenare (200 imagini/clasă), iar în acest scenariu inițializarea aleatoare poate duce la învățare instabilă și overfitting. Pentru a investiga dacă o inițializare mai bună ajută, am aplicat **fine-tuning** pornind de la un model CNN antrenat anterior pe **Imagebits**.

**Notă importantă (compatibilitate).** Pentru a evita mismatch între capete (număr diferit de clase) și eventuale diferențe, am încărcat **doar partea de feature extractor** (`features.*`) din checkpoint-ul Imagebits și am reinițializat head-ul (`classifier`) pentru cele 10 clase din Land Patches.

#### Checkpoint folosit:

- `cnn_imagebits_64.pth`
- 

### Procedură folosită (2 faze)

#### 1. Faza 1 — head only (feature extractor înghețat)

- `features` freeze (`requires_grad=False`)
- antrenez doar `classifier`
- `epochs=5`, `LR=1e-3`, `weight_decay=1e-4`

#### De ce?

Această fază adaptează rapid stratul final la noile clase fără a „distruge” caracteristicile deja învățate în convoluții.

#### 2. Faza 2 — full fine-tuning (unfreeze tot, LR mai mic)

- `features` unfreeze (`requires_grad=True`)
- fine-tuning pe tot modelul
- `epochs=10`, `LR=1e-4`, `weight_decay=1e-4`

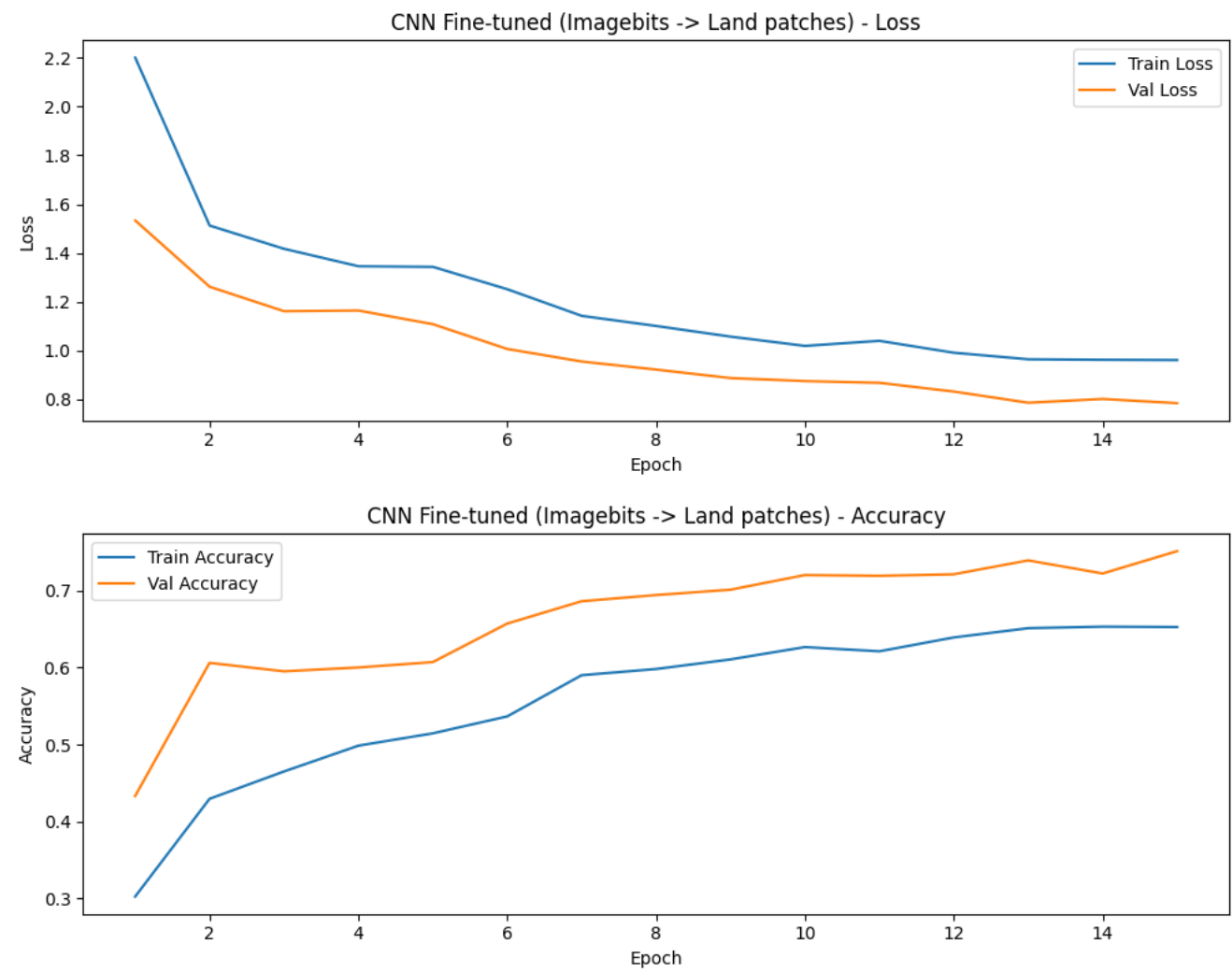
#### De ce LR mai mic?

După ce head-ul s-a adaptat, folosim un pas mai mic pentru a ajusta fin reprezentările fără schimbări bruște (mai stabil în prezența *domain gap*-ului).

---

### Curbe train/val (interpretare)





Ce se observă în curbe:

- **Val accuracy este mai mare decât train accuracy pe aproape tot parcursul.** Acest lucru este așteptat aici deoarece antrenarea folosește `train_loader_aug` (date augmentate → sarcină mai dificilă), iar val nu are augmentări. Prin urmare, modelul poate părea “mai bun” pe val decât pe train.
- **Train loss și val loss scad gradual,** fără oscilații mari, indicând o convergență relativ stabilă în cele două faze.
- Nu se vede un gap major train–val care să sugereze overfitting sever, însă performanța finală pe test rămâne limitată de dificultatea task-ului și de *domain gap*.

Rezultate pe test (Accuracy & Macro-F1)

Tabel rezultate (Fine-tuned CNN, test):

| Model          | Init                 | Aug (train) | Test Accuracy | Test Macro-F1 |
|----------------|----------------------|-------------|---------------|---------------|
| CNN fine-tuned | Imagebits (features) | yes         | 0.7354        | 0.7336        |

Comparativ cu celelalte experimente:

- față de **CNN augmented (from scratch)**: rezultate similare / ușor mai bune (0.7354 vs 0.7319)
- față de **CNN baseline (no aug)**: rămâne sub baseline (0.7354 vs 0.7689)

Acest rezultat sugerează că transferul din Imagebits oferă un start util (mai ales pentru stabilitate și `best_val_acc`), însă nu depășește cea mai bună soluție *from scratch* fără augmentări pe test (posibil din cauza diferențelor de domeniu între imagini naturale și imagini satelitare).

Classification report (Fine-tuned CNN, test)

Pe clase

| Clasă                | Precision | Recall | F1-score | Support |
|----------------------|-----------|--------|----------|---------|
| AnnualCrop           | 0.8827    | 0.7529 | 0.8126   | 700     |
| Forest               | 0.8462    | 0.8329 | 0.8395   | 700     |
| HerbaceousVegetation | 0.6571    | 0.6214 | 0.6388   | 700     |
| Highway              | 0.4020    | 0.3429 | 0.3701   | 700     |

| Clasă         | Precision | Recall | F1-score | Support |
|---------------|-----------|--------|----------|---------|
| Industrial    | 0.9188    | 0.8886 | 0.9034   | 700     |
| Pasture       | 0.6939    | 0.8000 | 0.7432   | 700     |
| PermanentCrop | 0.6188    | 0.6400 | 0.6292   | 700     |
| Residential   | 0.8363    | 0.9557 | 0.8920   | 700     |
| River         | 0.5804    | 0.6500 | 0.6132   | 700     |
| SeaLake       | 0.9186    | 0.8700 | 0.8936   | 700     |

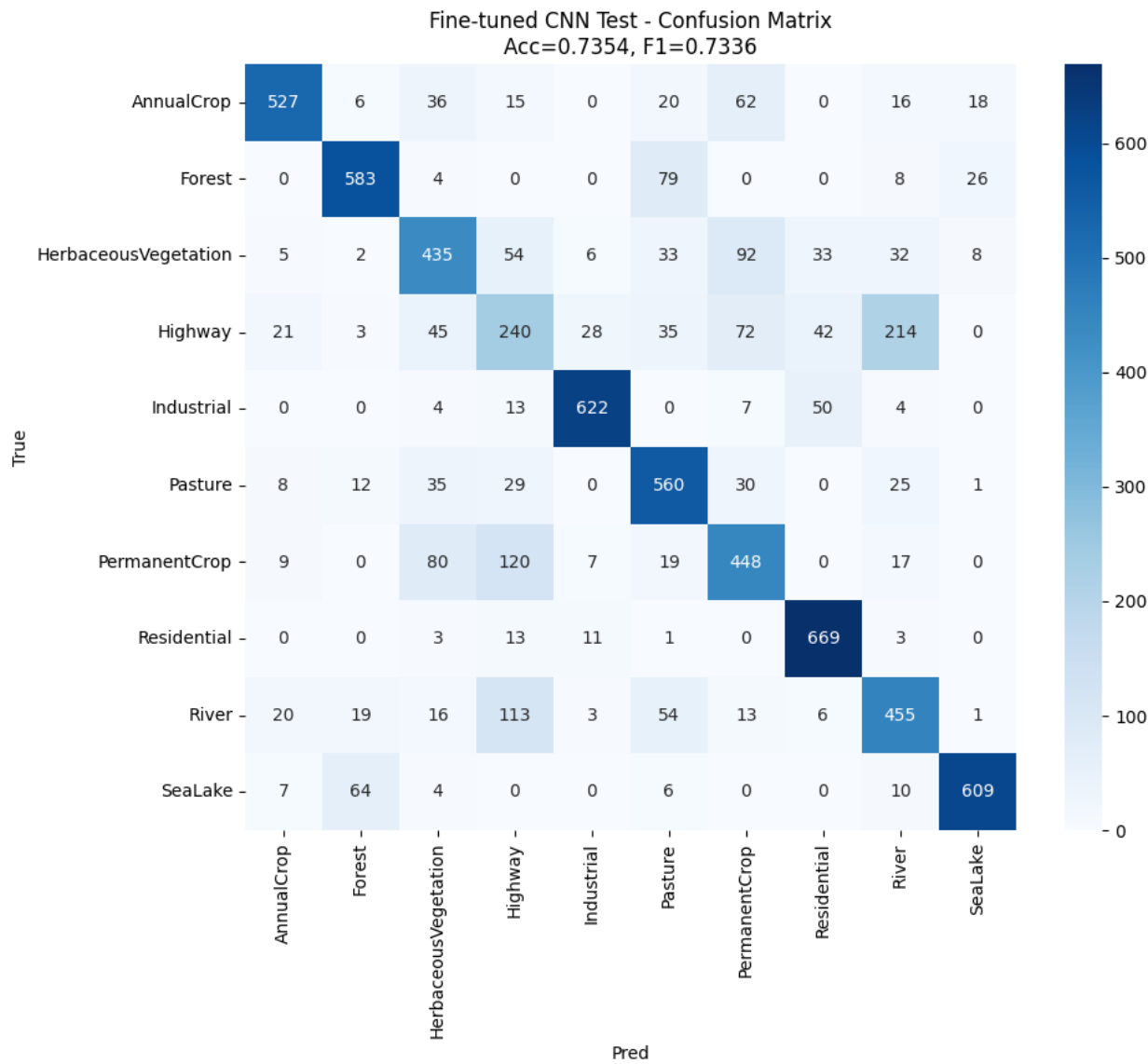
Agregat

| Metrică      | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| Accuracy     |           |        | 0.7354   | 7000    |
| Macro avg    | 0.7355    | 0.7354 | 0.7336   | 7000    |
| Weighted avg | 0.7355    | 0.7354 | 0.7336   | 7000    |

Comentarii / interpretare:

- Clase cu performanță ridicată: **Industrial**, **Residential**, **SeaLake** rămân foarte solide (recall mare), ceea ce indică faptul că trăsăturile învățate în backbone pot transfera bine pentru clase cu pattern-uri distinctive.
- Clasa **Highway** rămâne cea mai dificilă (recall ~0.34), sugerând că structurile liniare și contextul limitat la 64×64 sunt în continuare o sursă majoră de confuzie.
- **Pasture** și **PermanentCrop** au rezultate moderate; confuziile între clase de vegetație/agricultură nu sunt eliminate complet prin fine-tuning.

Matricea de confuzie (Fine-tuned CNN, test)



Ce evidențiază matricea:

- Diagonala este puternică pentru **Industrial**, **Residential**, **SeaLake**, confirmând scorurile ridicate din report.
- Confuzia majoră persistă pentru **Highway** (ex. predicții către **River** sau clase construite), ceea ce explică recall-ul scăzut.

Concluzie (Exp. 4)

Fine-tuning-ul din Imagebits către Land Patches oferă o inițializare non-aleatoare și o convergență stabilă (mai ales în combinație cu augmentările pe train), însă nu depășește CNN baseline fără augmentări pe test. Un motiv probabil este *domain gap*-ul (imagini naturale vs satelit), împreună cu faptul că CNN baseline deja generalizează bine pe distribuția Land Patches fără a necesita transfer.

Tabel comparativ — configurații & metrice (Land Patches)

| Model                           | Arh | Aug (train) | Optimizer                   | LR          | Batch | Epochs | Weight decay | Best Val Acc | Test Acc | Test Macro-F1 |
|---------------------------------|-----|-------------|-----------------------------|-------------|-------|--------|--------------|--------------|----------|---------------|
| MLP baseline                    | MLP | no          | Adam                        | 1e-3        | 64    | 20     | 1e-4         | 0.4990       | 0.4573   | 0.4371        |
| CNN baseline (no aug)           | CNN | no          | Adam                        | 1e-3        | 64    | 20     | 1e-4         | 0.7830       | 0.7689   | 0.7656        |
| CNN augmented                   | CNN | yes         | Adam                        | 1e-3        | 64    | 20     | 1e-4         | 0.7720       | 0.7319   | 0.7307        |
| CNN fine-tuned (from Imagebits) | CNN | yes         | Adam (head 1e-3; full 1e-4) | 1e-3 / 1e-4 | 64    | 5 + 10 | 1e-4         | 0.7510       | 0.7354   | 0.7336        |

Observații (rezumat):

- Trecerea de la **MLP** → **CNN** aduce un salt major (CNN baseline obține ~0.77 acc și ~0.77 macro-F1), confirmând importanța exploatarei structurii spațiale (convoluții + pooling + BN).
- În această rulare, **augmentarea** îmbunătățește dinamica de antrenare (regularizare), dar nu depășește CNN baseline pe test (posibil: augmentări ușor prea agresive / mismatch față de distribuția de test).
- **Fine-tuning** (init din Imagebits) nu depășește CNN baseline pe test; un motiv plauzibil este *domain gap* (imagini naturale vs satelit), plus sensibilitate la setări/augmentări.

Tabel detalii arhitecturale (opțional, pentru claritate)

| Model                           | Arch details (rezumat)                                                                                             | Dropout   |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------|
| MLP baseline                    | Flatten → Linear(512) → ReLU → Dropout(0.3) → Linear(128) → ReLU → Dropout(0.2) → Linear(10)                       | 0.3 / 0.2 |
| CNN baseline (no aug)           | Conv(3→32)→BN→ReLU→Pool; Conv(32→64)→BN→ReLU→Pool; Conv(64→128)→BN→ReLU→Pool; Flatten → FC(256) → Dropout → FC(10) | 0.4       |
| CNN augmented                   | Same CNN; train aug: HFlip/VFlip/Rot(10)/ColorJitter                                                               | 0.4       |
| CNN fine-tuned (from Imagebits) | Same CNN; init features din Imagebits; head reinit; freeze→unfreeze                                                | 0.4       |

Concluzii finale — Land Patches

Pe setul **Land Patches (64×64, support redus la train: 200 imagini/clasă)**, arhitectura a avut impactul dominant asupra performanței. **MLP baseline** a obținut rezultate semnificativ mai slabe (Test Acc ≈ **0.46**, Macro-F1 ≈ **0.44**), iar curbele Train vs Val au indicat un **gap** vizibil și o generalizare modestă. Acest lucru este coerent cu limitarea MLP-ului: prin **aplatizarea** imaginii, pierde structura spațială, iar pentru un task bazat pe **texturi/pattern-uri** (terenuri/scene) această informație este esențială.

Trecerea la un **CNN cu Conv + BatchNorm + MaxPool** a dus la un salt major de performanță. **CNN baseline (fără augmentări)** a fost cea mai bună configurație pe test (Test Acc ≈ **0.77**, Macro-F1 ≈ **0.77**), confirmând utilitatea *inductive bias*-ului convoluțional pentru imagini. Analiza pe clase (classification report / matricea de confuzie) arată performanță ridicată pentru clase cu pattern-uri distinctive (ex. **Forest**, **Residential**, **Industrial**, **SeaLake**), în timp ce clase precum **Highway** și **PermanentCrop** rămân mai dificile — explicabil prin **ambiguități de textură** și context limitat la 64×64.

Augmentarea (flip H/V, rotații mici, ColorJitter) a acționat ca regularizare și a schimbat dinamica de antrenare (train mai greu, tendință mai mică de supraînvățare), însă în această rulare **nu a depășit** CNN baseline pe test (Test Acc ≈ **0.73**, Macro-F1 ≈ **0.73**). Un motiv plauzibil este că augmentările pot fi ușor prea agresive pentru distribuția de test sau necesită tuning suplimentar (intensitate, LR/scheduler, epoci) pentru a se transforma într-un câștig consistent.

**Fine-tuning-ul din Imagebits** → **Land Patches** (încărcare doar a **features.\***, reinițializare head, freeze→unfreeze cu LR mai mic în faza 2) a oferit rezultate comparabile cu CNN augmented (Test Acc ≈ **0.74**, Macro-F1 ≈ **0.73**), dar **nu a depășit** CNN baseline. Acest lucru sugerează existența unui *domain gap* între imagini naturale (Imagebits) și imagini satelitare (Land Patches), iar pentru această arhitectură simplă, antrenarea directă pe Land Patches rămâne cea mai eficientă.

**Concluzie:** pentru Land Patches, alegerea arhitecturii (CNN vs MLP) este factorul decisiv, iar cea mai bună configurație în experimentele de față este **CNN baseline (no aug)**; augmentarea și fine-tuning-ul au oferit regularizare / stabilitate, dar fără un câștig final pe test în această setare.

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |

# RoSent — Analiză de sentiment (RNN & LSTM) + Augmentare text

## RoSent — Dataset & EDA

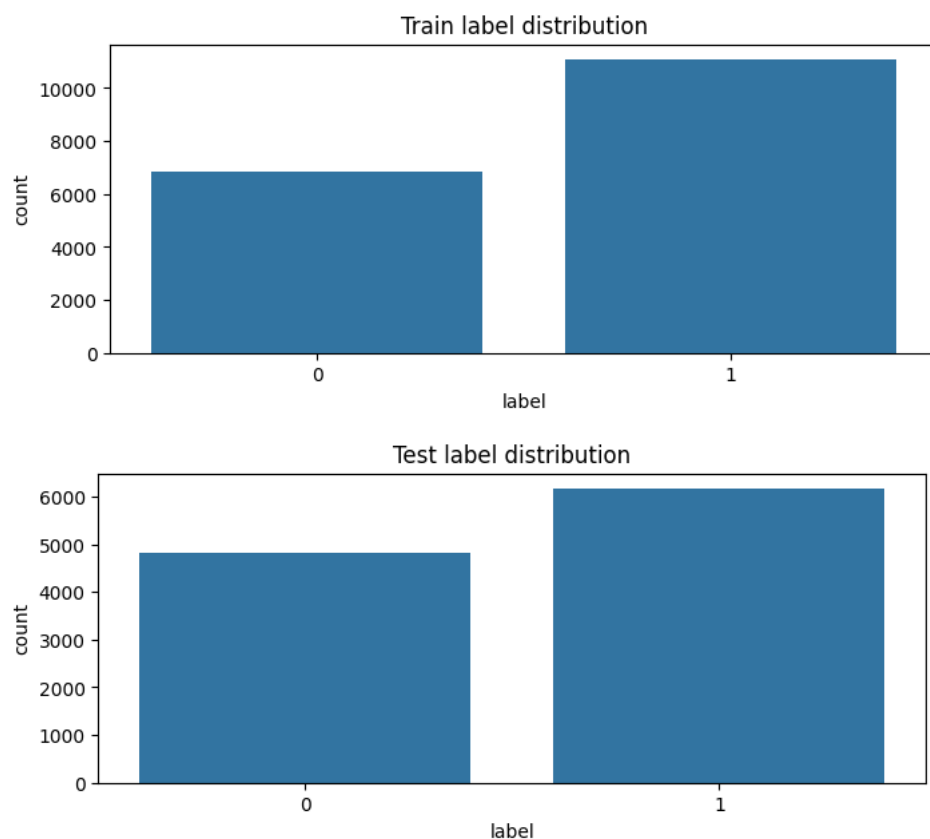
Setul **RoSent** conține texte în limba română (propoziții/recenzii) etichetate binar pentru **sentiment** (`label ∈ {0,1}`); dacă etichetele sunt string-uri de tip **negative/positive**, acestea sunt mapate la `0/1`. Datele sunt încărcate direct din CSV (train/test), iar setul de train este împărțit în **train/val** folosind **StratifiedShuffleSplit**, astfel încât proporțiile claselor să rămână similare în ambele subset-uri.

### Dimensiuni:

- **Train inițial:** 17941 exemple
- **Split Train/Val:** 15249 train / 2692 val (`test_size=0.15`, stratificat)
- **Test:** 11005 exemple

### Distribuția etichetelor (imbalanțe ușor)

Din distribuțiile pe train/test se observă un **dezechilibru moderat**: clasa 1 are mai multe exemple decât clasa 0 (aceiași tipar apare și în test: `support(0)=4828`, `support(1)=6177`). Acest aspect este important deoarece un model poate obține o acuratețe aparent bună favorizând clasa majoritară; din acest motiv, pe lângă **Accuracy**, raportăm și **Macro-F1**, care penalizează performanța slabă pe o clasă chiar dacă cealaltă clasă este bine clasificată.

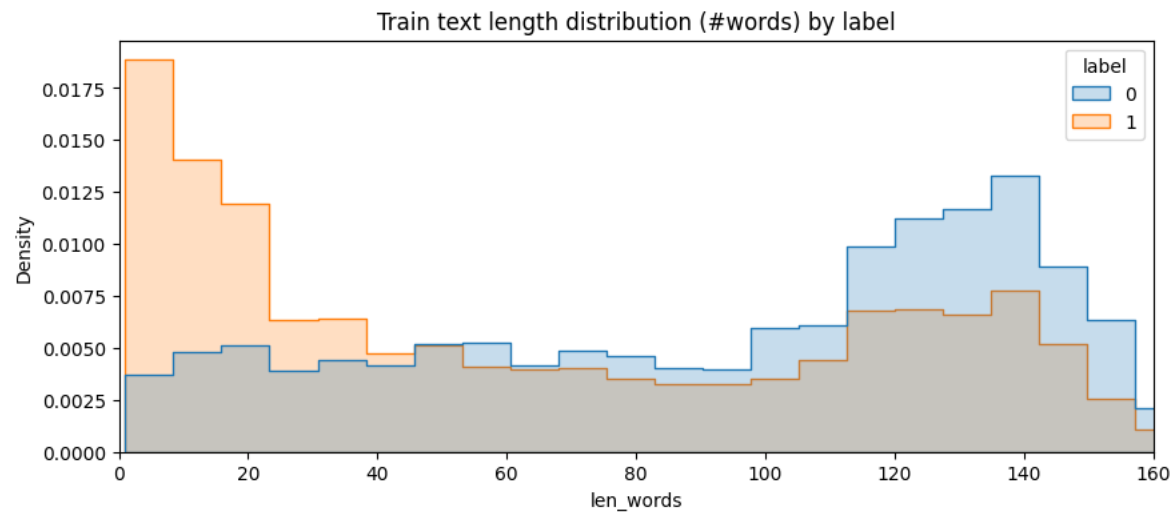


### Lungimea textelor (variabilitate intra-clasă)

Am analizat lungimea textelor măsurată ca **număr de cuvinte** (`len_words`). Distribuția este **largă**, cu multe texte scurte, dar și o coadă de texte lungi (review-uri). În plus, există o variabilitate semnificativă în interiorul fiecărei clase (textele pot fi foarte scurte sau destul de lungi indiferent de label), ceea ce sugerează că sentimentul poate fi exprimat atât concis ("excelent"), cât și argumentat (review lung).

Percentilii pe train indică faptul că majoritatea exemplelor sunt acoperite cu o lungime relativ moderată:

- mediană: 77 cuvinte
- p90: 142 cuvinte
- p95: 148 cuvinte
- p99: 160 cuvinte



Decizie de preprocesare (justificare):

Am setat `MAX_LEN = 128` ca limită de secvență, deoarece acoperă o mare parte din distribuție (aprox. până între mediană și p90), reducând masiv padding-ul și costul de antrenare. Textele mai lungi sunt **trunchiate**, iar cele mai scurte sunt **padded** la aceeași dimensiune, pentru a putea folosi batch-uri eficiente.

Cuvinte frecvente (observații)

Ca o verificare simplă, am extras cele mai frecvente cuvinte per clasă după o curățare minimă (lowercase, eliminare URL, păstrare caractere românești). Listele sunt dominate de cuvinte funcționale (“de”, “și”, “a”, “nu”, “în”), iar apariția frecventă a lui “**nu**” în ambele clase sugerează că semnalul de sentiment nu depinde doar de prezența unui singur cuvânt, ci de **context** (negări, intensificatori, expresii). Asta motivează folosirea unor modele secvențiale (RNN/LSTM) care pot surprinde ordinea și contextul local al cuvintelor.

| Rank | Label 0 – Word | Count | Label 1 – Word | Count |
|------|----------------|-------|----------------|-------|
| 1    | de             | 27325 | de             | 29147 |
| 2    | si             | 17233 | si             | 22580 |
| 3    | a              | 15270 | a              | 15257 |
| 4    | nu             | 13939 | este           | 14261 |
| 5    | sa             | 13254 | in             | 13888 |
| 6    | in             | 12336 | o              | 12207 |
| 7    | ca             | 11934 | sa             | 12010 |
| 8    | este           | 11709 | un             | 11901 |
| 9    | o              | 10856 | ca             | 11610 |
| 10   | un             | 10691 | nu             | 10427 |
| 11   | film           | 9033  | film           | 9597  |
| 12   | care           | 7766  | care           | 8957  |
| 13   | mai            | 7568  | mai            | 8627  |
| 14   | cu             | 7045  | la             | 8546  |
| 15   | la             | 6985  | cu             | 8511  |
| 16   | am             | 6778  | am             | 8197  |
| 17   | acest          | 6571  | pe             | 6840  |
| 18   | fost           | 6387  | pentru         | 6419  |
| 19   | pentru         | 6235  | acest          | 6178  |
| 20   | pe             | 6165  | foarte         | 6023  |

Legătura cu task-ul și cu alegerile ulterioare

Concluziile EDA au influențat direct designul experimentelor:

- **Imbalance moderat** → raportăm **Macro-F1** pe lângă Accuracy și analizăm diferențele între clase.
- **Texte cu lungime variabilă** → alegem `MAX_LEN=128` pentru un compromis între informație păstrată și eficiență.
- **Semnal dependent de context** (ex. negări) → testăm modele secvențiale (RNN și LSTM, inclusiv bidirecțional).
- **Generalizare** → explorăm augmentări simple la nivel de token, aplicate doar pe train, pentru a crește robustețea la variații de formulare.

---

## Preprocesare (NLP) & reprezentare

### Curățare + tokenizare

Pentru a reduce zgomotul din text și a obține o reprezentare cât mai consistentă, am aplicat o **curățare minimă** înainte de tokenizare:

- conversie la **lower-case** (reduce variația artificială între „Bun” și „bun”);
- eliminare **URL-uri** (nu conțin în general semnal relevant pentru sentiment și pot introduce tokeni rari);
- păstrarea caracterelor alfabetice, inclusiv **diacriticele românești** (ăâîșț/șț) și a spațiilor;
- normalizare de whitespace (multiple spații → un singur spațiu).

După curățare, tokenizarea este realizată astfel:

- dacă **spaCy** este disponibil, folosim `spacy.blank("ro")` (tokenizare robustă, fără a depinde de un model mare);
- altfel, folosim un fallback simplu cu `split()`, pentru a păstra pipeline-ul funcțional indiferent de mediu.

#### Justificare:

Tokenizarea și curățarea consistente sunt importante pentru construirea vocabularului și pentru a evita fragmentarea inutilă a aceluiași cuvânt în forme diferite (ex. variații de caz, caractere speciale). În plus, păstrarea diacriticelor este utilă în română, deoarece poate diferenția cuvinte și forme relevante semantic.

### Split train/val (stratificat) + vocabular

Pentru a evalua corect generalizarea modelului pe parcursul antrenării, am separat setul de antrenare inițial într-un split **train/val** folosind `StratifiedShuffleSplit` (`test_size=0.15`, `random_state=42`). Stratificarea păstrează proporția claselor în ambele subset-uri, ceea ce este important în prezența unui dezechilibru (chiar și moderat), deoarece altfel setul de validare ar putea conține prea multe exemple dintr-o singură clasă și ar distorsiona metricile.

După definirea split-ului, am construit vocabularul **exclusiv din train** (`df_tr`) pentru a evita *data leakage* (informații din val/test care influențează reprezentarea). Vocabularul este obținut prin numărarea frecvențelor tuturor tokenilor din train și păstrarea doar a celor cu frecvență `>= MIN_FREQ`.

- Split: `train=15249`, `val=2692` (stratificat, `test_size=0.15`)
- Vocabular: construit **doar din train**
- Prag frecvență: `MIN_FREQ = 2` (filtrează tokeni rari/typo-uri și reduce dimensionalitatea)
- Tokeni speciali:
  - `<pad>` pentru padding (folosit ca să aducem toate secvențele la aceeași lungime)
  - `<unk>` pentru tokeni necunoscuți (cuvinte noi în val/test care nu apar în train)
- Dimensiune vocabular: `VOCAB_SIZE = 25773`
- ID-uri speciale: `pad_id = 0`, `unk_id = 1`

#### Justificare `MIN_FREQ=2`:

Tokenii care apar o singură dată sunt adesea zgomot (typo, nume proprii rare, forme foarte specifice) și cresc artificial vocabularul. Filtarea lor reduce riscul de overfitting și face embedding-ul mai "curat" și mai eficient.

#### Justificare `<unk>`:

În NLP, este normal ca în val/test să apară cuvinte nevăzute în train. Maparea lor la `<unk>` permite modelului să rămână robust și să nu "crape" la inferență, în timp ce păstrează posibilitatea de a învăța pattern-uri bazate pe contextul restului propoziției.

### Encoding + padding / trunchare

După construirea vocabularului, fiecare text este transformat într-o secvență de **ID-uri** folosind maparea `token → index` din `vocab`. Tokenii care nu există în vocabular (cuvinte nevăzute în train) sunt mapați la `unk_id`, astfel încât pipeline-ul să rămână robust la inferență.

Pentru a putea folosi batch-uri eficiente în PyTorch, toate secvențele trebuie să aibă aceeași lungime. De aceea, fiecare secvență este adusă la o dimensiune fixă `MAX_LEN` prin:

- **trunchiere**: dacă textul este mai lung decât `MAX_LEN`, păstrăm doar primii `MAX_LEN` tokeni;
- **padding**: dacă textul este mai scurt, adăugăm tokenul `<pad>` până la lungimea `MAX_LEN`.

În implementare, acest lucru este realizat de funcția `encode()`:

- `ids = [vocab.get(t, unk_id) for t in toks]` (mapare token → id, fallback la `<unk>`)
- `ids = ids[:MAX_LEN]` (trunchiere)
- `ids = ids + [pad_id] * (MAX_LEN - len(ids))` (padding)

#### Justificare `MAX_LEN = 128`:

- Din EDA, mediană  $\approx 77$  cuvinte și  $p90 \approx 142$ , deci `128` acoperă o mare parte din distribuție fără a introduce prea mult padding.
- Limitează costul computațional (timp/memorie) și stabilizează antrenarea.
- Trunchierea afectează doar textele foarte lungi; pentru task-ul de sentiment, semnalul este de multe ori prezent relativ devreme în text, iar LSTM/BiLSTM poate capta context suficient în această fereastră.

#### Notă practică:

În sanity check, un exemplu encodat are lungime `128`, iar primele valori sunt ID-uri valide din vocabular, confirmând că pipeline-ul de codificare produce secvențe cu dimensiunea așteptată.

---

## Setup de antrenare (comun)

- Loss: **CrossEntropyLoss** (clasificare binară, 2 clase; logits → softmax intern în loss)
- Optimizator: **Adam**
  - `LR = 1e-3` (din `CFG.LR`)
  - `weight_decay = 1e-4` (din `CFG.WEIGHT_DECAY`, regularizare L2)
  - Scheduler: **nu** (menținut constant pentru a compara corect experimentele)
- Batch size: `BATCH_SIZE = 64` (din `CFG.BATCH_SIZE`)
- Epoci: `EPOCHS = 8` (din `CFG.EPOCHS`)
- Device: **cuda** dacă este disponibil, altfel **cpu** (din `CFG.DEVICE`)
- Reproducibilitate: `SEED = 42` (`set_seed(42)`)
- Metrice raportate: **Accuracy** și **Macro-F1**  
(Macro-F1 este relevant în prezența unui dezechilibru moderat între clase, pentru evaluare echilibrată)
- Evaluare val/test: **fără augmentare** (comparabilitate și măsurare corectă a generalizării)

## Modele utilizate (RNN vs LSTM)

### Cum funcționează un RNN

Un **RNN** procesează secvența token cu token și menține o **stare ascunsă** care „rezumă” contextul precedent.

Avantaje: simplu, rapid; dezavantaje: poate avea dificultăți în captarea dependențelor mai lungi (mai ales față de LSTM).

#### RNNClassifier:

- `Embedding(vocab_size, EMB_DIM=300)`
- `nn.RNN(hidden_dim=128, num_layers=1, bidirectional=False)`
- head: `Dropout(0.2) → Linear(128, 2)`

### Cum funcționează un LSTM

Un **LSTM** este un RNN îmbunătățit cu mecanisme de tip *gates* (input/forget/output) care controlează ce informație se păstrează/uită, fiind mai robust la dependențe mai lungi.

Varianta **bidirecțională (BiLSTM)** citește secvența în ambele direcții și concatenează reprezentările, util când semnalul de sentiment poate depinde de cuvinte apărute atât la început, cât și la final.

#### LSTMClassifier:

- `Embedding(vocab_size, EMB_DIM=300)`
- `nn.LSTM(hidden_dim=128, num_layers=1, bidirectional=True)`
- head: `Dropout(0.3) → Linear(256, 2)` (2×128 datorită bidirecționalității)

## Augmentare text (experiment dedicat)

Pentru a testa dacă putem îmbunătăți generalizarea pe RoSent fără a schimba arhitectura, am introdus augmentări simple la nivel de token, aplicate **doar pe train** cu probabilitate `AUG_PROB = 0.3` (din `CFG.AUG_PROB`). În fiecare apel de augmentare se alege aleator una dintre cele trei operații:

- **Random swap:** se aleg două poziții aleatorii și se inversează tokenii (1 swap).
- **Random delete:** fiecare token este șters cu probabilitate `p = 0.1` (cu grijă să nu rămână secvența goală).
- **Random insert:** se inserează un token existent (duplicare) într-o poziție aleatorie (1 insert).

#### Justificare (problemă → alegere → efect așteptat):

- **Problemă:** modelele secvențiale pot supraînvața formulări specifice și ordinea exactă a cuvintelor (mai ales cu vocab mare). Pe RoSent, aceeași polaritate poate fi exprimată în multe feluri, iar un model care învață „șabloane” prea rigide generalizează mai slab.
- **Alegere:** swap/delete/insert introduc perturbări mici ale secvenței, forțând modelul să fie mai robust la variații locale (ordinea tokenilor, cuvinte lipsă/extra) fără a schimba drastic textul.
- **Efect așteptat:** reducerea overfitting-ului și o performanță mai bună pe val/test (în special la o arhitectură simplă precum RNN).

**Notă:** augmentările sunt aplicate **numai pe train**; val/test rămân ne-augmentate pentru o evaluare stabilă și comparabilă.

### Exemplu de augmentare

**Original:** micul oras californian diablo este afectat de decese misterioase dupa ce seriful robert lopez descopera o cutie veche. l

**Augmented:** micul oras californian descopera diablo este afectat de decese misterioase dupa ce seriful robert lopez descopera o cuti

Exemplul confirmă că augmentarea produce o propoziție foarte similară ca vocabular și sens general, dar cu o mică modificare locală (ex. repoziționarea unui cuvânt), ceea ce este exact tipul de variație pe care dorim să-l învățăm ca invariance.

### Pipeline (implementare)

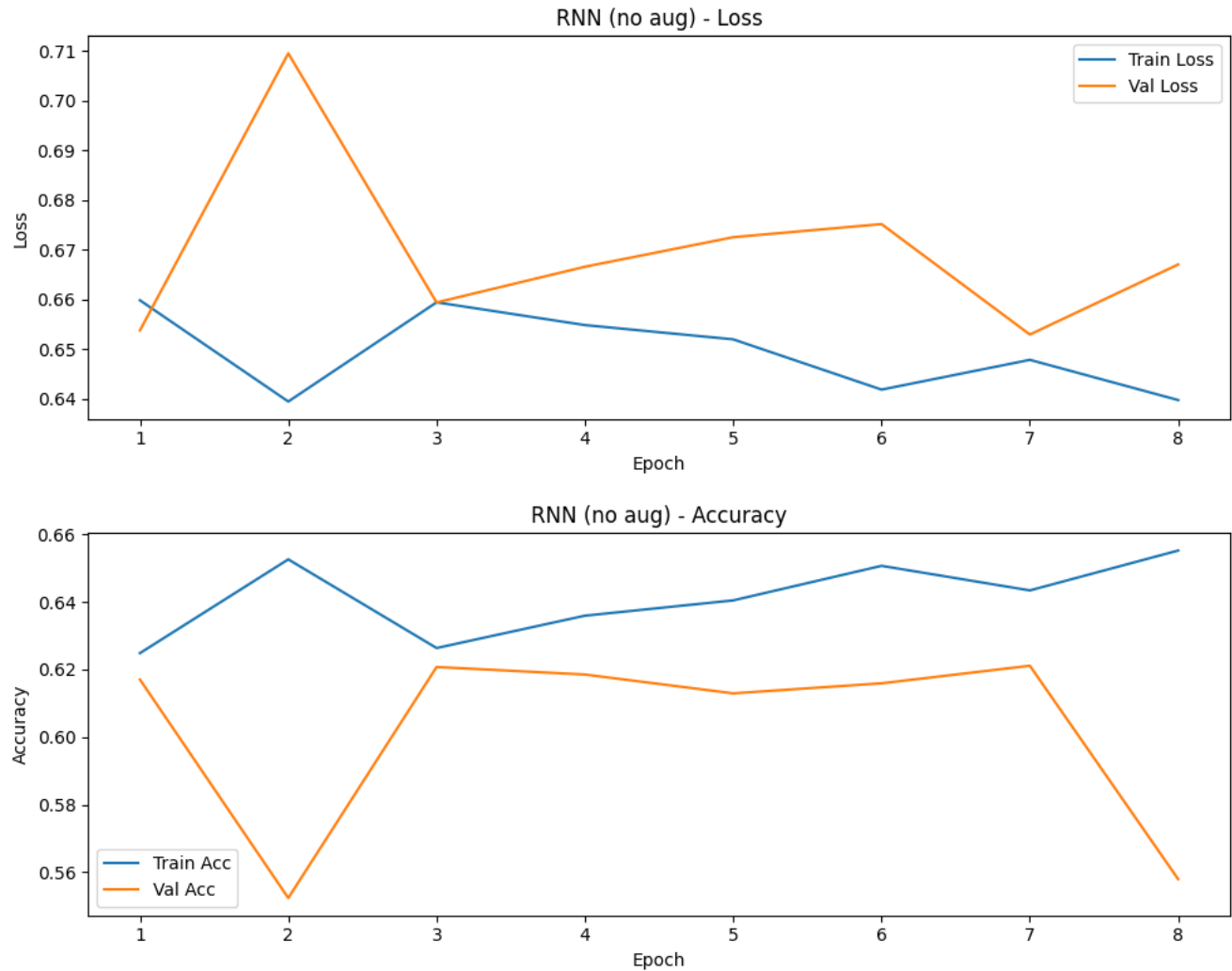
Datele sunt încărcate cu un `Dataset` PyTorch care tokenizează textul și returnează perechi (x, y), iar batching-ul se face prin `DataLoader` (shuffle doar la train). Opțional aplicăm augmentare stocastică pe text în timpul antrenării, cu probabilitatea `CFG.AUG_PROB`. Pentru reprezentarea tokenilor folosim `nn.Embedding`, inițializat fie trainabil, fie din vectori FastText, iar clasificarea se face cu modele RNN/LSTM (opțional bidirecționale) urmate de dropout și un strat liniar.

## Experimente

Exp. 1 — RNN simplu (fără augmentare)

Setare experiment

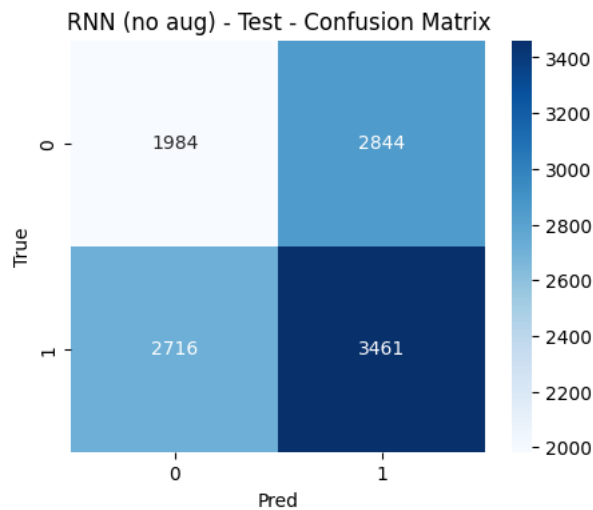
- **Model:** `RNNClassifier` (hidden=128, layers=1, bidirectional=False, dropout=0.2)
- **Embedding:** `nn.Embedding` (trainabil) / FastText dacă este activat în config
- **Augmentare:** NU
- **Optimizare:** Adam (lr=`CFG.LR`), weight\_decay=`CFG.WEIGHT_DECAY`
- **Epoci:** `CFG.EPOCHS`, batch\_size=`CFG.BATCH_SIZE`



Rezultate pe setul de test

| Metrică       | Valoare |
|---------------|---------|
| Test Accuracy | 0.4948  |
| Test Macro-F1 | 0.4855  |





Classification report (test)

| Clasă        | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.4221    | 0.4109 | 0.4165   | 4828    |
| 1            | 0.5489    | 0.5603 | 0.5546   | 6177    |
| Accuracy     |           |        | 0.4948   | 11005   |
| Macro avg    | 0.4855    | 0.4856 | 0.4855   | 11005   |
| Weighted avg | 0.4933    | 0.4948 | 0.4940   | 11005   |

Interpretare (pe baza graficelor + CM)

1) Curbele de loss (train vs val).

Train loss scade ușor, semn că modelul învață tipare din datele de antrenare. Val loss este însă mai instabilă și rămâne, în general, mai mare decât train loss, ceea ce indică **generalizare slabă** și posibil început de **overfitting/instabilitate** după câteva epoci.

2) Curbele de accuracy (train vs val).

Train accuracy urcă treptat ( $\approx 0.62 \rightarrow \approx 0.65$ ), pe când val accuracy rămâne în jur de  $\approx 0.61$  și fluctuează. Diferența train–val sugerează un **generalization gap**: modelul se potrivește mai bine pe train decât pe date nevăzute.

3) Val vs Test (de ce cade pe test).

Deși val accuracy este  $\approx 0.61$ , pe test obținem **0.4948**, ceea ce poate indica:

- **distribution shift** între val și test (texte mai diferite ca stil/vocabular);
- sau evaluarea pe test a fost făcută pe ultimul epoch, nu pe checkpoint-ul cu cel mai bun scor pe validare (în prezența fluctuațiilor pe val, alegerea epocii poate conta mult).

4) Analiza erorilor (Confusion Matrix).

Modelul confundă frecvent clasele în ambele sensuri ( $0 \rightarrow 1$  și  $1 \rightarrow 0$ ), iar raportul pe clase arată că recunoaște mai bine **clasa 1** ( $\text{Recall}=0.5603$ ) decât **clasa 0** ( $\text{Recall}=0.4109$ ). Asta indică un comportament ușor înclinat spre prezicerea clasei 1 și o separare slabă între clase.

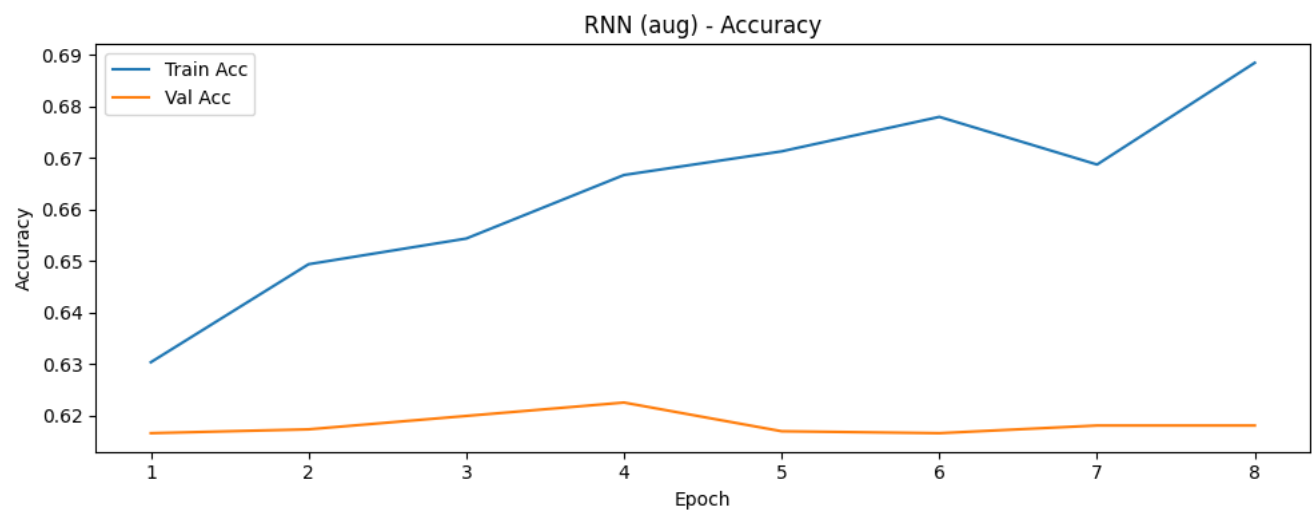
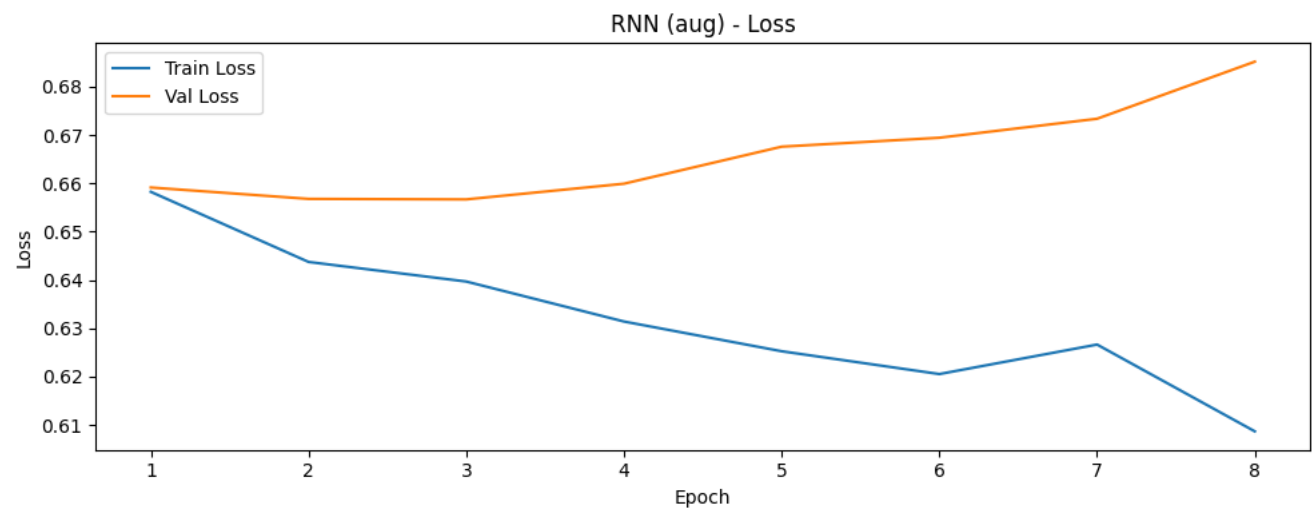
**Concluzie:** RNN-ul simplu, fără augmentare și fără mecanisme mai expresive (LSTM/GRU, bidirecționalitate, pooling/attention), are dificultăți să surprindă variațiile lingvistice și dependențele mai lungi, ducând la **Macro-F1 modest (0.4855)** și performanță scăzută pe test.

Spike-uri/variații bruște pe validare.

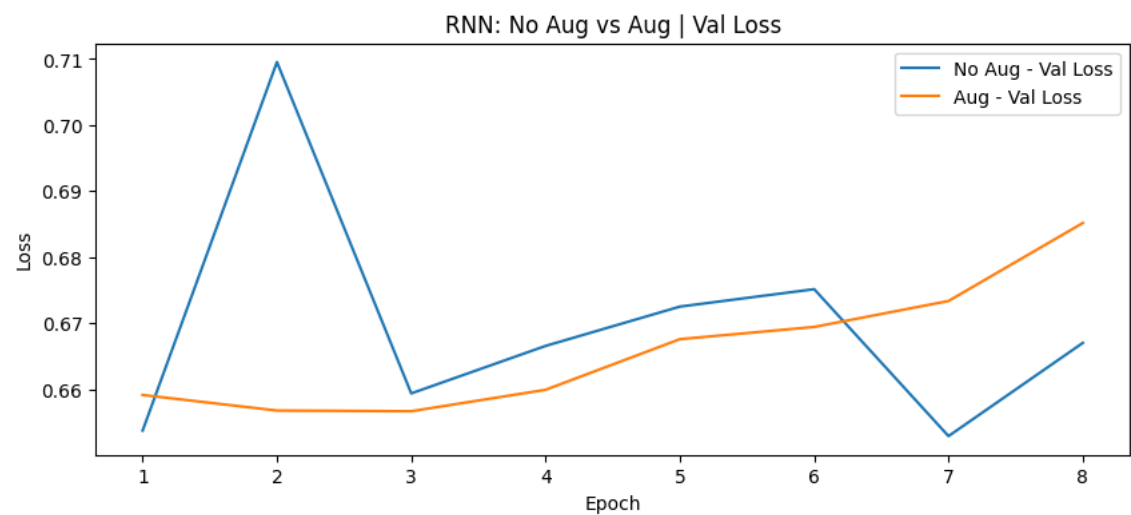
Curbele de validare prezintă oscilații “triunghiulare” (salt brusc într-o epocă urmat de revenire). Acest tip de comportament indică o estimare instabilă a performanței pe validare și poate fi cauzat de pași de optimizare prea mari (learning rate), sensibilitatea unui RNN simplu la variații ale greutăților și/sau lipsa mecanismelor de stabilizare (ex. gradient clipping). În plus, dacă evaluarea nu este făcută în mod determinist (ex. `model.eval()`), dropout-ul poate introduce variații suplimentare.

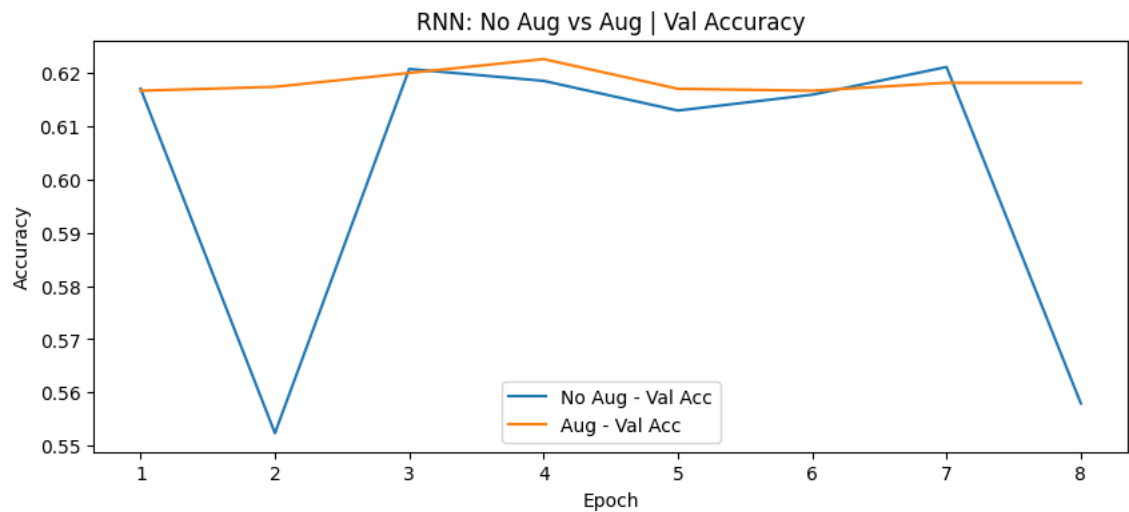
Exp. 2 — RNN simplu (cu augmentare) + comparație directă

Grafice (train vs val)



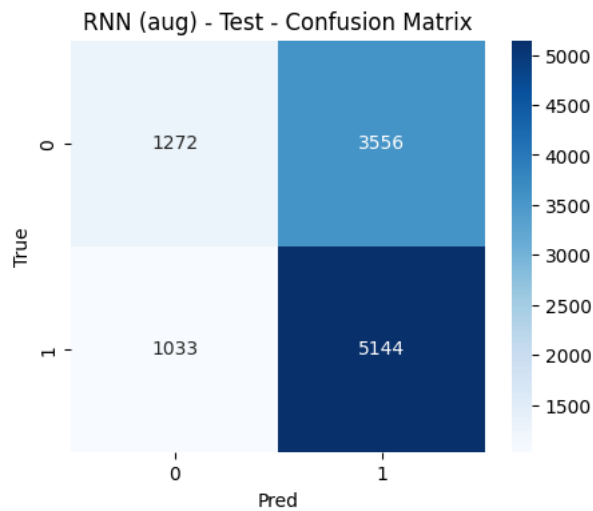
Comparație augmentare vs fără augmentare (pe validare)





Rezultate pe setul de test

| Metrică       | Valoare |
|---------------|---------|
| Test Accuracy | 0.5830  |
| Test Macro-F1 | 0.5241  |



Classification report (test)

| Clasă        | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.5518    | 0.2635 | 0.3567   | 4828    |
| 1            | 0.5913    | 0.8328 | 0.6915   | 6177    |
| Accuracy     |           |        | 0.5830   | 11005   |
| Macro avg    | 0.5716    | 0.5481 | 0.5241   | 11005   |
| Weighted avg | 0.5740    | 0.5830 | 0.5446   | 11005   |

Interpretare (pe baza graficelor + comparației + CM)

- 1) Efectul augmentării în curbele train/val.**  
Cu augmentare, **train accuracy crește constant** (până spre ~0.69), în timp ce **val accuracy rămâne aproape plată** (~0.616–0.623). În paralel, **train loss scade**, dar **val loss crește treptat** pe epoci. Acest tipar indică faptul că modelul se potrivește tot mai bine pe datele de antrenare (chiar augmentate), însă nu traduce proporțional în îmbunătățiri pe validare — un semn de **generalizare limitată** pentru un RNN simplu.
- 2) Compararea directă No Aug vs Aug (graficele de validare).**  
Pe validare, varianta cu augmentare are curbe **mai stabile** (fără căderea puternică observată la No Aug), iar nivelul de val accuracy este comparabil sau ușor mai bun în majoritatea epocilor. Totuși, val loss pentru Aug tinde să urce spre final, sugerând că după un anumit punct ar fi util **checkpoint/early stopping** (alegerea epocii cu cel mai bun scor pe validare).

3) De ce testul crește, dar Macro-F1 rămâne modest.

Pe test, augmentarea crește **Accuracy** (0.5830 vs 0.4948) și **Macro-F1** (0.5241 vs 0.4855). Îmbunătățirea există, dar Macro-F1 rămâne relativ mic deoarece performanța este **puternic dezechilibrată între clase**.

4) Asimetrie clară pe clase (CM + raport).

Modelul devine mult mai „agresiv” pe **clasa 1**:

- clasa 1: **Recall=0.8328** (prinde majoritatea exemplelor 1)
- clasa 0: **Recall=0.2635** (ratează multe exemple 0)

Matricea de confuzie arată foarte multe **0 → 1** (false positives), ceea ce explică F1 foarte mic pentru clasa 0 (0.3567), chiar dacă acuratețea totală rămâne rezonabilă. Practic, augmentarea pare să ajute modelul să recunoască mai bine clasa 1, dar **nu îl ajută suficient să separe corect clasa 0**, ceea ce trage în jos Macro-F1.

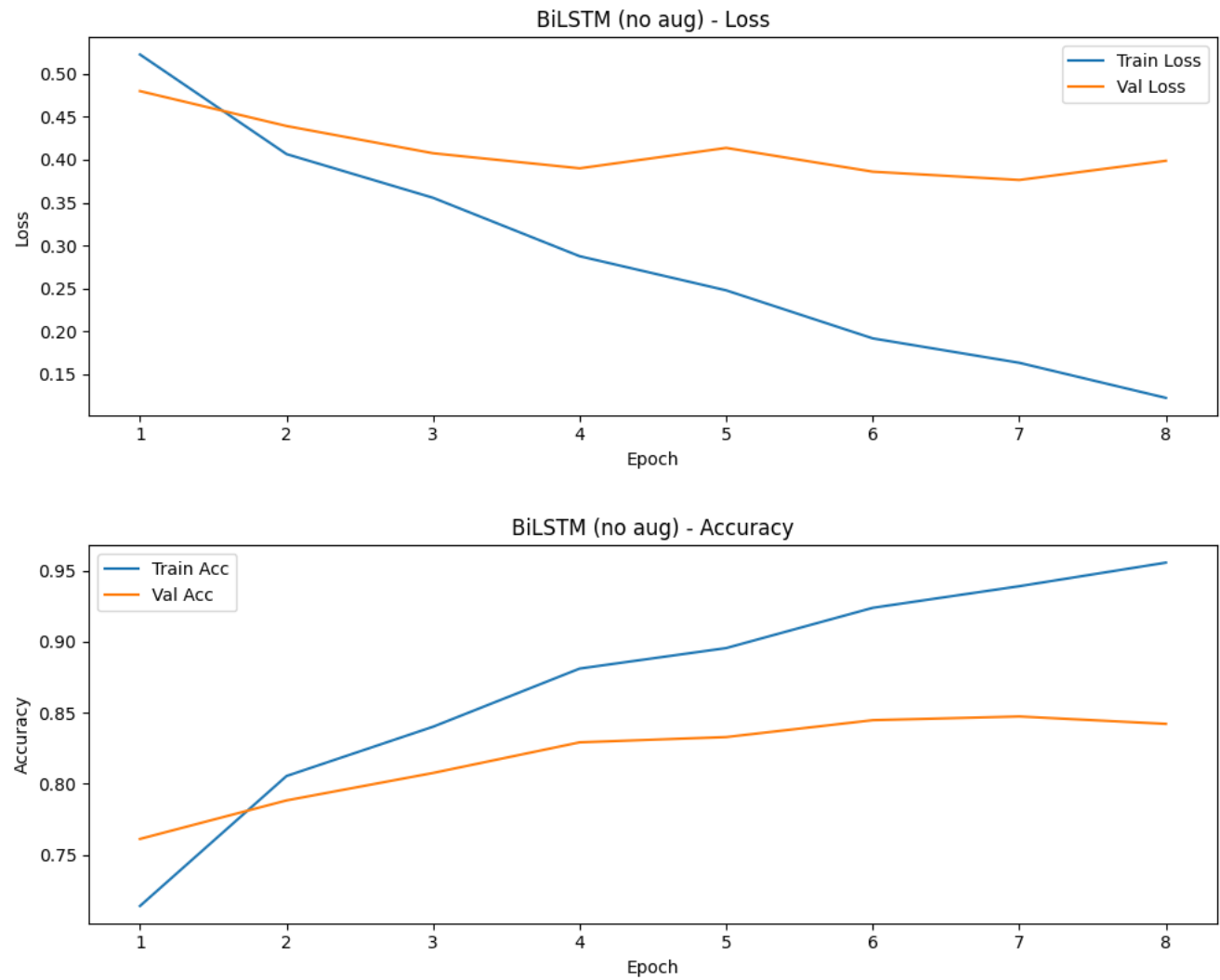
**Concluzie:** augmentarea îmbunătățește performanța globală pe test și stabilizează validarea față de varianta fără augmentare, dar introduce/întărește o **tendință de a prezice clasa 1**, ducând la recall foarte mic pe clasa 0. Pentru a corecta acest comportament, ar merita testate în experimente următoare: ponderarea claselor (class weights), threshold tuning și/sau un model mai expresiv (LSTM/biLSTM).

Exp. 3 — BiLSTM (bidirecțional, fără augmentare) + evaluare

De ce schimbarea față de RNN

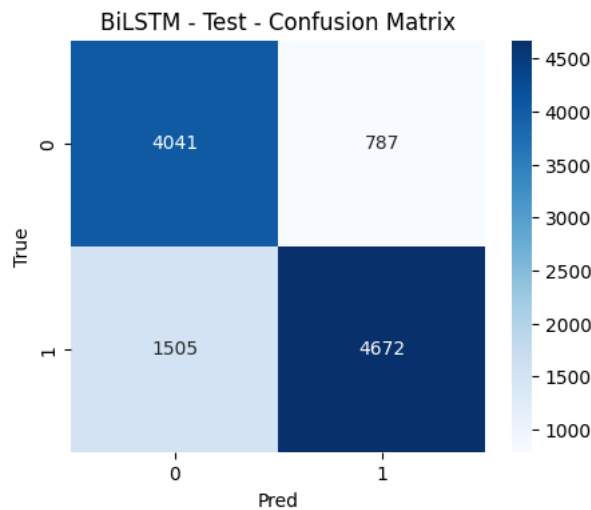
Un **LSTM** are porți (gates) care controlează ce informație se păstrează/uită, deci gestionează mai bine dependențe mai lungi (ex. negații, relații între cuvinte la distanță). Varianta **bidirecțională** (BiLSTM) citește secvența atât stânga→dreapta, cât și dreapta→stânga, capturând mai bine contextul complet al propoziției.

Grafice



Rezultate pe setul de test

| Metrică       | Valoare |
|---------------|---------|
| Test Accuracy | 0.7917  |
| Test Macro-F1 | 0.7910  |



Classification report (test)

| Clasă        | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.7286    | 0.8370 | 0.7791   | 4828    |
| 1            | 0.8558    | 0.7564 | 0.8030   | 6177    |
| Accuracy     |           |        | 0.7917   | 11005   |
| Macro avg    | 0.7922    | 0.7967 | 0.7910   | 11005   |
| Weighted avg | 0.8000    | 0.7917 | 0.7925   | 11005   |

Interpretare (bazată pe grafice + CM)

1) Loss: învățare clară + ușor overfitting spre final.

Train loss scade puternic (modelul învață bine), iar val loss scade în primele epoci, apoi are o mică revenire/oscilație spre final. Acest tipar arată că BiLSTM generalizează mult mai bine decât RNN-ul simplu, dar după un punct poate apărea **overfitting**, deci ar fi utilă alegerea "best epoch" după val loss/val acc (checkpoint/early stopping).

2) Accuracy: val urcă și apoi plafonează.

Train accuracy crește continuu (până ~0.95), în timp ce val accuracy crește până în zona ~0.84–0.85 și apoi se stabilizează ușor (chiar cu o mică scădere la final). Diferența train–val (gap) indică din nou un început de overfitting, dar nivelul absolut pe val este mult mai bun decât la RNN.

3) Confusion matrix: mult mai echilibrat decât la RNN.

Din CM:

- **TN (0→0): 4041** și **TP (1→1): 4672** (corecte multe pe ambele clase)
- **FP (0→1): 787** și **FN (1→0): 1505**

Spre deosebire de RNN (mai ales cu augmentare, care împingea masiv spre clasa 1), aici erorile sunt mai "controlate". Se observă totuși o asimetrie: modelul are **recall mai mare pe clasa 0** (0.8370) decât pe clasa 1 (0.7564), adică ratează mai des exemple din clasa 1 decât din clasa 0.

4) Concluzie:

BiLSTM aduce un salt major (Acc≈0.79, Macro-F1≈0.79) deoarece poate modela mai bine contextul și dependențele lungi. Curbele sugerează că ar merge și mai bine cu **checkpoint pe epoca cu val loss minim / val acc maxim** și eventual regularizare suplimentară (ex. early stopping, dropout tuning).

Tabel comparativ — setup + metrice (rezumat experimente)

| Model           | Arch   | hidden_dim | layers | bi-dir | dropout | LR    | wd     | batch | epochs | MAX_LEN | EMB_DIM | MIN_FREQ | AUG_PROB | Best Val Acc |
|-----------------|--------|------------|--------|--------|---------|-------|--------|-------|--------|---------|---------|----------|----------|--------------|
| RNN (no aug)    | RNN    | 128        | 1      | False  | 0.2     | 0.001 | 0.0001 | 64    | 8      | 128     | 300     | 2        | 0.3      | 0.6211       |
| RNN (aug)       | RNN    | 128        | 1      | False  | 0.2     | 0.001 | 0.0001 | 64    | 8      | 128     | 300     | 2        | 0.3      | 0.6226       |
| BiLSTM (no aug) | BiLSTM | 128        | 1      | True   | 0.3     | 0.001 | 0.0001 | 64    | 8      | 128     | 300     | 2        | 0.3      | 0.8473       |

## Interpretare tabel comparativ

### 1) Câștigul augmentării pe RNN (Exp. 1 → Exp. 2).

Comparând cele două RNN-uri (aceeași arhitectură și hiperparametri, diferența fiind augmentarea), se vede un progres clar pe test:

- **Test Acc:** 0.4948 → 0.5830 (+0.0882)
- **Test Macro-F1:** 0.4855 → 0.5241 (+0.0386)

Asta sugerează că augmentarea a crescut robustetea modelului la variații de formulare și a redus parțial "căderea" pe test. Totuși, îmbunătățirea e limitată de capacitatea redusă a unui RNN simplu și de tendința lui de a rămâne instabil/biased pe clase (cum se vede și din rapoartele pe clase și CM).

### 2) Saltul major adus de BiLSTM (Exp. 3).

BiLSTM depășește net ambele RNN-uri:

- față de RNN (aug):
  - **Test Acc:** 0.5830 → 0.7917 (+0.2087)
  - **Macro-F1:** 0.5241 → 0.7910 (+0.2669)
- față de RNN (no aug): creșterea este și mai mare.

Explicația principală este că LSTM (mai ales bidirecțional) modelează mai bine **dependențe pe termen lung și contextul complet al propoziției**, ceea ce este esențial în analiză de sentiment (negații, contraste, "dar", contexte târzii etc.). Rezultatul Macro-F1 ridicat arată și o clasificare mai echilibrată între clase decât la RNN (în special decât la RNN cu augmentare).

### 3) Best Val Acc vs Test Acc (diferențe și ce înseamnă).

La RNN-uri, Best Val Acc este ~0.62, dar testul poate fi mult mai jos (mai ales la no-aug), ceea ce indică **generalizare slabă** și posibil **diferență de distribuție** între val și test sau sensibilitate la epoca aleasă.

La BiLSTM, Best Val Acc este mult mai mare (0.8473) și se traduce într-un test puternic (0.7917), deci există o corelație mai bună între validare și generalizare.

### 4) Concluzie generală.

- **Augmentarea** ajută atunci când modelul este slab (RNN), dar nu rezolvă limitările arhitecturii și poate menține bias pe clase.
- **Schimbarea arhitecturii** (BiLSTM) aduce cea mai mare îmbunătățire: atât Acc, cât și Macro-F1 cresc semnificativ, ceea ce indică o generalizare mai bună și o separare mai clară între clase.

## Concluzie finală — RoSent

Pe RoSent, **BiLSTM (bidirecțional)** este modelul cu cea mai bună performanță, atingând **Test Acc = 0.7917** și **Test Macro-F1 = 0.7910**. Comparativ, **RNN (no aug)** obține **Acc = 0.4948 / Macro-F1 = 0.4855**, iar **RNN (aug)** urcă la **Acc = 0.5830 / Macro-F1 = 0.5241**. Prin urmare, cea mai mare îmbunătățire nu vine din "tuning" minor, ci dintr-o arhitectură capabilă să modeleze mai bine contextul.

Rezultatele indică faptul că, pentru analiza de sentiment, **captarea dependențelor și a contextului global** (inclusiv prin **bidirecționalitate**) este esențială: BiLSTM reușește să mențină performanță ridicată și relativ echilibrată între clase, ceea ce se reflectă în Macro-F1 mare. În schimb, augmentarea textuală simplă (swap/delete/insert) ajută RNN-ul față de baseline, dar rămâne limitată de capacitatea unui RNN simplu și poate amplifica o **asimetrie între clase** (modelul ajunge să favorizeze mai mult clasa 1, cu recall ridicat pe 1 și recall scăzut pe 0), ceea ce reduce scorul Macro-F1 în raport cu un model mai robust.

În ansamblu, în aceste experimente, cea mai eficientă strategie a fost trecerea de la RNN la **BiLSTM**, care oferă generalizare net superioară și o separare mai bună între clase.